Lecture 2: Language Modeling: From Counts to Neural Networks

A language model assigns probabilities to strings. This lecture builds language models first by counting and then with neural networks, and explains why counting, which worked for forty years, ran out.

Two ideas older than neural networks run through it.

Text as a random source. Andrey Markov in 1913 and Claude Shannon in 1948 treated text as the output of a random process whose statistics can be counted and whose predictability can be measured. Counting which word follows which context gives the n-gram models that ran speech recognition and machine translation for decades. Their obstacle: almost every long context is new, and a table of counts has nothing to say about a new context.

Words as points in a space. A word is characterized by the company it keeps, so words used in similar contexts can be placed near each other in a vector space. These **word embeddings** are among the most widely used ideas in machine learning, well beyond language modeling: they power semantic search, vector databases, and recommendation systems. They can be built by counting or by prediction, and the two routes meet.

Neural language models join the two ideas: they predict the next token, as n-gram models do, but from learned vectors, so that what is learned about one context carries over to similar ones. The lecture builds them in increasing order of complexity, each removing the main limitation of the one before: the bigram model written as a neural network, which learns by gradient descent the table that counting gives; the **fixed-window model** of Bengio et al. (2003), which predicts the next word from learned vectors of the previous few; the **recurrent neural network** (RNN), which carries a summary of the whole prefix in a vector; and the **long short-term memory** network (LSTM), whose gated memory lets it use information from far back in the prefix.

The lecture has three parts:

1. counting (§§1–4): text as a random source, n-gram models, measuring a language model, and smoothing;
2. word embeddings (§5): why vectors, where they are used, their history, vectors from counts and from prediction (word2vec), and why the two agree;
3. neural language models (§§6–8): the bigram model as a network, the fixed-window model, recurrent networks, and the LSTM, each with how it is trained and how it generates.

Parts 2 and 3 are the core. Part 1 sets up the object, the vocabulary, and the yardstick, and is kept short: the details of smoothing (§A.2, §A.3) and hidden Markov models, the other main tool of the count era (§A.6), are in the appendix.

The approach follows Andrej Karpathy's *Neural Networks: Zero to Hero*, which builds language models from scratch in code, trains them, and samples from them; the course is built in the same spirit. His two *makemore* lectures cover §6 for names, one character at a time, and his essay *The Unreasonable Effectiveness of Recurrent Neural Networks* (2015) is the model for §§7–8.

1. Text as a random source

Markov's count

In 1913 Andrey Markov used Pushkin's *Eugene Onegin* to make a point about probability (Markov, 1913; English translation 2006). He wanted to show that the law of large numbers does not need independent trials: it also holds for a sequence in which each trial depends on the one before, what is now called a **Markov chain**. Text was his example of a source that is random but dependent.

He reduced the first 20,000 letters of the poem to vowels and consonants. 1,104 pairs of consecutive letters were both vowels, where independent letters would give about 3,730. A letter depends on the one before: after a vowel, a vowel follows with probability 0.128; after a consonant, with probability 0.663. The chain built from these two numbers then predicted, roughly, a statistic it was not fitted to: how much the number of vowels varies from one block of 100 letters to the next. A chain on the previous two letters predicted it better (§A.1 tells the story in full).

This is the idea the first part of the lecture follows: treat text as a random process, estimate the probability of the next symbol given the previous ones by counting, and test the model against text it was not fitted to. Markov's chain is the first language model.

Shannon's approximations

Claude Shannon's *A Mathematical Theory of Communication* (1948) took the same view: a source of messages is a random process, its predictability is measured by its entropy, and the more of its statistics a model captures, the more the model's samples resemble the source. He showed this with samples from successively better approximations to English: letters chosen uniformly (zero order), independently with their frequencies (first order), given the previous letter (second order) or the previous two (third order), and words given the previous word.

The experiment is easy to repeat on TinyStories (Eldan and Li, 2023), short stories in simple English written by a language model, which is the corpus of every measurement in these notes and of the course code. Its 21,990 stories are split at random into 17,592 for training, 2,199 for validation, and 2,199 for testing (§3 explains why). Every experiment here can be rerun in the Week 2 notebook, which the lecture alternates with; its Part 1 repeats this one.

Below are samples from models that choose each character given the previous $n-1$, with probabilities equal to relative frequencies in the training stories (§2). The order n counts the chosen character together with its context, as in Shannon's numbering: $n = 1$ draws each character independently with its frequency, and the uniform row is the zero-order approximation.

Order	Sample
uniform	rt"jzysmxpgzblb?jcm?,ox?pungxvj' .s yoin,fhicv..nzy?mlaoglv.rnilefe"jx,.bc".skigtkvyp.nayofi
$n = 1$	ari ef o eheiowamna wactifp oe nu,laoo.smnseot sef nataiel! ooit,s wllwrr d .t agswo.hn nd
$n = 2$	mes pasth anoele s heded junt hend. sed. shed nopomas tow. the, birm thed t n rto st iskeird
$n = 3$	saw heir he up boalk. stim. her frod their wore peethe warna bra wit witchose pone he frien.
$n = 5$	once upon a tim. the pot. john and warned the cool and yield. at to be and puts tom gentle b
$n = 7$	once upon a time for you are going to eat her smile. then finally, there to find a secret. s

and from models that choose each word given the previous $n-1$ words (the uniform row draws every word of the vocabulary with the same probability):

Order	Sample
uniform	savored soldier billy's long whenever truthful sideways obstacles tony reaching imagined warning elle mischievous elves
$n = 1$	over she breath lily wanted toys . see learned lived was the was because very , strawberries
$n = 2$	once outside during their songs . " thank you know how fast and pockets . he couldn't sleep
$n = 3$	once upon a time , let's have a problem . she laughed , and sing songs . she had made a new one .

Local regularities appear first: spelling by about $n = 3$ for characters, common phrases for words. Sentences that hold together over more than a few words appear at none of these orders. Shannon's 1951 paper, *Prediction and Entropy of Printed English*, measured predictability directly by having people guess the next letter: about one bit per letter (Lecture 1, Section 6).

Tokens: characters, words, or pieces

Before text can be modeled it is cut into a sequence of **tokens** from a finite **vocabulary** $\mathcal{V}$ by a **tokenizer**. Shannon used two units, letters and words, and the choice matters (Figure 1).

- **Characters.** Lowercased, TinyStories uses 33 characters: 26 letters, the space, and six punctuation marks. Nothing is ever unknown, but sequences are long (867 characters per story on average), and a model must learn spelling before meaning.
- **Words.** Sequences are about four times shorter (201 words per story) and each token carries meaning, but the vocabulary is large and grows with the corpus. Here it has the 8,679 words that occur at least twice in training, an end-of-story token, and an unknown-word token `<unk>` for every rarer word (0.2% of test words): 8,681 tokens.
- **Pieces of words.** Production models use subword tokens cut by frequency: **frequent words stay whole, and rare words are split into pieces.** GPT-2's tokenizer has a fixed vocabulary of 50,257 tokens whose pieces go down to single bytes, so any text can be written and nothing is unknown. On the TinyStories vocabulary (Figure 1b) it keeps 97% of the words that occur 1,000 times or more in training as one token, but only 53% of those that occur fewer than ten times: *happy* is one token, *pebble* three (*pe*, *b*, *ble*). A common word costs one prediction, a rare word a few, and a text is only

slightly longer in tokens than in words. Byte-pair encoding, which learns such vocabularies, is part of Week 5.

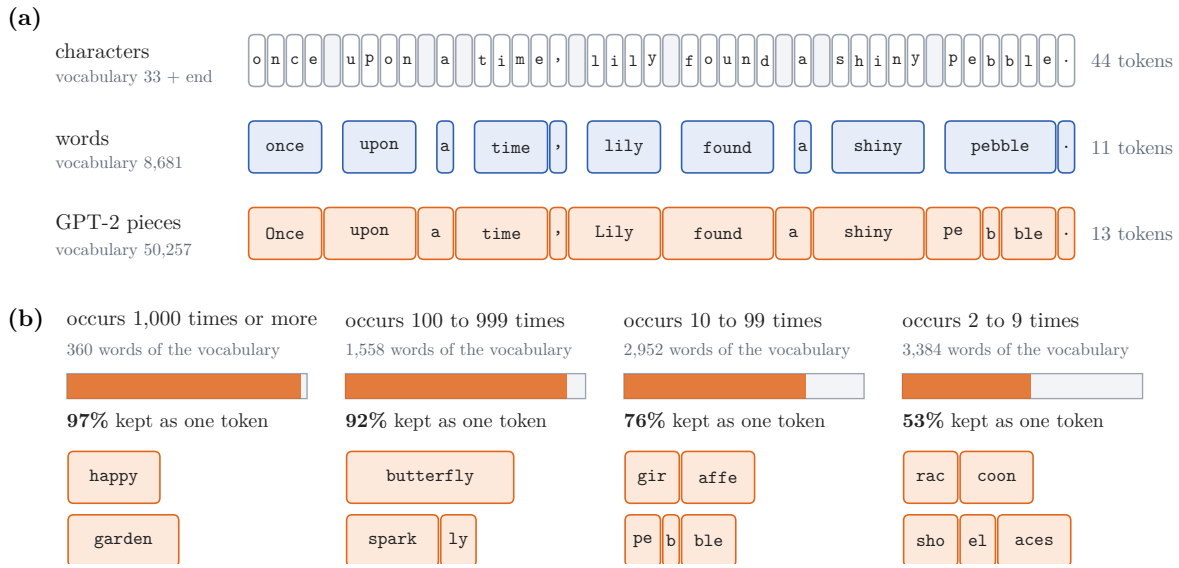


Figure 1. Tokens. (a) One sentence in three tokenizations: these notes' lowercased characters and words, and the tokenizer of GPT-2, which keeps capital letters and attaches the space before a word to the word. (b) Frequent words stay whole, rare words are split: the TinyStories vocabulary grouped by how often each word occurs in the training stories, the share that GPT-2's tokenizer keeps as one token, and two examples from each group. The tokenizer was built from web text, so it is frequency there that decides: "countries", which occurs seven times in TinyStories, is one token.

Each model in this lecture works with characters or words, for reasons given where the model appears:

Models	Token	Vocabulary	Context	Sections
Character n-grams	character	33 + end	$n-1$ characters	§1, §4
Word n-grams	word	8,681	$n-1$ words	§§2–4
Count vectors, word2vec	word	3,000; 6,453	two words on each side	§5
Bigram network	character, then word	34; 8,681	one token	§6
Fixed-window network	word	8,681	four words	§6
Recurrent network, LSTM	character, then word	34; 8,681	the whole prefix	§§7–8

The chain rule, BOS and EOS

Write a string as tokens $x_{1:T} = (x_1, \dots, x_T)$ from $\mathcal{V}$, whatever the tokens are. By the chain rule of probability,

$$q(x_{1:T}) = \prod_{t=1}^{T} q(x_t \mid x_{<t}),$$

where $x_{<t} = (x_1, \dots, x_{t-1})$ is the **prefix**. This is an identity, not an assumption. A model that supplies the conditionals $q(\cdot \mid x_{<t})$ and generates by sampling from them in order is **autoregressive**. Every model in this lecture is; they differ only in how they compute $q(\cdot \mid x_{<t})$: from a table of counts, from a fixed window of the prefix, or from a summary of the whole prefix.

Strings have different lengths, so the model must also decide when to stop. An **end-of-sequence** token EOS is predicted like any other token and ends the string. The probability of a complete string is

$$q(x_{1:T}, \text{EOS}) = \left[\prod_{t=1}^{T} q(x_t \mid x_{<t})\right] q(\text{EOS} \mid x_{1:T}).$$

Without the last factor, the product is the probability that a sampled string *begins* with $x_{1:T}$, so *the cat* and *the cat sleeps* would compete for the same mass. A **beginning-of-sequence** token BOS is

prepended as the context of the first prediction and is never predicted. (Conditionals that rarely choose EOS can put probability on strings that never end; §A.4 gives a condition that rules this out.)

Picture all strings as a tree of prefixes, a **trie**: the root is the empty prefix, the children of a node are its extensions by one token, and an EOS leaf ends a string. A language model labels the branches out of each node with $q(\cdot \mid \text{prefix})$, which sum to one, and a complete string's probability is the product of the labels on its path to its EOS leaf. Strings of different lengths end at different depths (Figure 2a). An autoregressive model is a rule for the branch probabilities at every node of this infinite tree, and the models of this lecture differ in how much of the path to a node the rule reads.

2. n-gram language models

The Markov assumption

The number of possible prefixes grows exponentially with their length. An **n-gram model** assumes that only the last $n-1$ tokens matter:

$$q(x_t \mid x_{<t}) \approx q(x_t \mid x_{t-n+1:t-1}).$$

A unigram model ($n = 1$) ignores the past, a bigram model ($n = 2$) conditions on one previous token, a trigram model on two. Missing context at the start is filled with BOS: a trigram model predicts the first word after (BOS, BOS) and the second after (BOS, x_1). This is Markov's chain with words for letters, whose state is the last $n-1$ tokens. In the tree of §1, all nodes whose prefixes end in the same $n-1$ tokens get the same branch probabilities, so the infinite tree folds into a finite graph with one state per history (Figure 2b).

The parameters are the distributions $q(\cdot \mid h)$, one for each **history** h of $n-1$ tokens.

Maximum likelihood is counting

Let $c(h, w)$ be the number of times token w follows history h in the training corpus, and $c(h) = \sum_w c(h, w)$. The **likelihood** of a candidate model q is the probability it assigns to the training corpus, the product of $q(w \mid h)$ over the corpus's prediction events. Its logarithm groups by history:

$$\log \mathcal{L}(q) = \sum_h \sum_w c(h, w) \log q(w \mid h).$$

Maximum likelihood chooses the q that makes the training corpus most probable. Each history's distribution appears in its own term, so each is maximized separately: for a fixed h, maximize $\sum_w c(h, w) \log q_w$ subject to $\sum_w q_w = 1$. Setting the derivative of $\sum_w c(h, w) \log q_w - \lambda(\sum_w q_w - 1)$ to zero gives $q_w = c(h, w)/\lambda$, and the constraint gives $\lambda = c(h)$:

$$q_{\text{MLE}}(w \mid h) = \frac{c(h, w)}{c(h)}.$$

The estimate is the relative frequency. Tokens never seen after h get probability zero, and histories never seen get no estimate.

A worked example: four sentences

Take the corpus *the cat sleeps*, *the cat eats fish*, *the dog sleeps*, *a dog eats*, and a bigram model with vocabulary $\mathcal{V} = \{\text{the}, \text{a}, \text{cat}, \text{dog}, \text{sleeps}, \text{eats}, \text{fish}, \text{EOS}\}$. Every word and every EOS is a prediction event, seventeen in all. The counts are:

h	**the**	**a**	**cat**	**dog**	**sleeps**	**eats**	**fish**	**EOS**	$c(h)$
BOS	3	1	0	0	0	0	0	0	4
the	0	0	2	1	0	0	0	0	3
a	0	0	0	1	0	0	0	0	1
cat	0	0	0	0	1	1	0	0	2
dog	0	0	0	0	1	1	0	0	2
sleeps	0	0	0	0	0	0	0	2	2
eats	0	0	0	0	0	0	1	1	2
fish	0	0	0	0	0	0	0	1	1

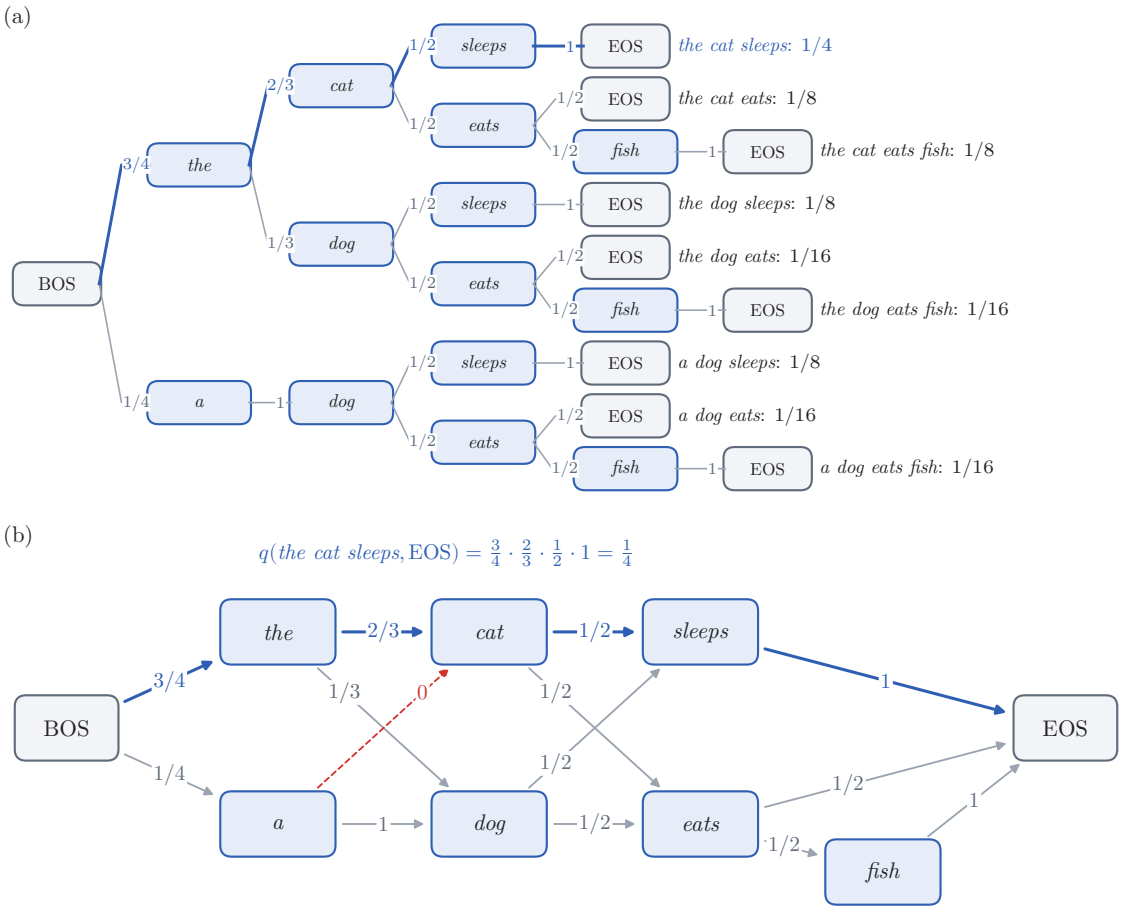


Figure 2. The bigram model of the four-sentence corpus. (a) As a tree of prefixes: every branch carries a maximum-likelihood probability $q(w \mid h)$, and every EOS leaf ends a complete sentence whose probability is the product along its path; the leaves sit at different depths because the sentences have different lengths, and the nine probabilities sum to one. (b) As a Markov chain: the tree's nodes that end in the same word share their branch probabilities and are merged into one state. The blue path is "the cat sleeps", with probability 1/4. The dashed red arrow, from "a" to "cat", was never observed, so its probability is zero.

Dividing each row by its total gives the model. Figure 2a draws it as a tree of prefixes: its nine leaves are the nine sentences the model can produce, of three or four words, and their probabilities sum to one.

Figure 2b folds the tree into a chain. The table answers three questions. **Scoring** multiplies the factors along a sentence:

$$q_{\text{MLE}}(\text{the cat sleeps}, \text{EOS}) = \frac{3}{4} \cdot \frac{2}{3} \cdot \frac{1}{2} \cdot 1 = \frac{1}{4}.$$

Prediction reads off a row: after *eats*, EOS and *fish* are equally likely. **Generation** samples a token from the row of the current history, makes it the new history, and repeats until EOS.

The tree also shows what EOS does. The prefix *the cat eats* has probability $\frac{3}{4} \cdot \frac{2}{3} \cdot \frac{1}{2} = \frac{1}{4}$, which the model splits between two complete sentences: *the cat eats* (EOS next) and *the cat eats fish*, $\frac{1}{8}$ each.

The table generalizes, in a limited way. *the dog eats fish* never occurred, but each of its transitions did, and it gets $\frac{3}{4} \cdot \frac{1}{3} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot 1 = \frac{1}{16}$. *a cat sleeps* is just as good a sentence, but *a cat* never occurred, so it gets probability zero. An n-gram model recombines pieces it has seen; it says nothing about a piece it has not.

Sparsity

More data does not remove the problem. With a vocabulary of K tokens a trigram model has K^3 parameters: 6.5×10^{11} for TinyStories' $K = 8{,}681$, against 3.6 million training tokens. The fraction of test n-grams that never occur in training is

	bigrams	**trigrams**	**4-grams**
unseen in training	3.8%	18.3%	41.1%

Language has a long tail: a large share of the n-gram types in any corpus occur once, and new text keeps producing new ones. So the choice of n is a trade-off. A longer history says more about the next word (after *a* almost any noun or adjective can follow; after *once upon a*, nearly always *time*), and with exact probabilities a longer history would never predict worse. But the probabilities are estimated from counts, and the longer the history, the fewer times it occurs in training, so its estimate rests on less evidence; most long histories occur once or never. Longer histories make the *true* conditional distributions sharper and their *estimates* poorer. Which effect wins cannot be read off samples; it needs a score, which §3 defines.

3. Measuring a language model

Samples suggest that a trigram model beats a bigram model, but they cannot settle the decisions ahead: which order n to use, and which way to fix the zero probabilities of §2 (the smoothing methods of §4). Samples are random, judging them is subjective, and a small difference would not show. We need one number, computed the same way for every model, that measures how well a model predicts text it was not trained on.

Log loss

A model that gives probability $q(x_t \mid x_{<t})$ to the token that actually occurs pays a **log loss** of $-\log q(x_t \mid x_{<t})$. In base 2 the loss is in bits: probability $1/2$ costs one bit, $1/8$ three bits, a certain prediction nothing, and probability zero an infinite amount (Figure 3). The loss of a text is the sum over its tokens, which by the chain rule is the negative log probability of the text. Lecture 1 (Section 5) read it as a code length; here it is a score, and lower is better.

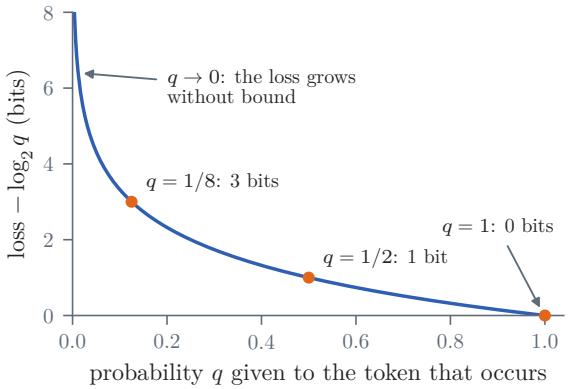


Figure 3. The log loss of one prediction as a function of the probability the model gave to the token that occurred. Halving the probability costs one more bit; a probability of zero costs an infinite amount.

On the training text, the total log loss is the negative log likelihood of §2, and the count table minimizes it. So a low training loss proves nothing: the table gives every unseen n-gram probability zero and pays an infinite loss on the first new n-gram of a new story, and a model of high enough order reproduces its training stories and has learned nothing beyond them. The score is therefore computed on **held-out** text, not used for estimation, and everything meant to make a model generalize, from smoothing (§4) to the restricted model families and regularization of neural networks (§6), is judged by its loss there.

Cross-entropy

If test text is drawn from a true distribution p, the model's expected loss per event is the **cross-entropy**

$$H(p, q) = -\sum_x p(x) \log q(x) = H(p) + D_{\mathrm{KL}}(p \,\|\, q),$$

where $H(p) = -\sum_x p(x) \log p(x)$ is the entropy of the source and $D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log(p(x)/q(x))$ is the Kullback–Leibler divergence, which is never negative and is zero only if $q = p$. The entropy is a property of the text that no model can beat; the rest is the model's error. Minimizing expected log loss minimizes $D_{\mathrm{KL}}(p\|q)$, and maximum likelihood (§2) is the same minimization with the training text in place of p: training and evaluation use one objective on different text. For sequences the floor is the entropy rate of the source, $\lim_{T\to\infty} H(X_{1:T})/T$.

Perplexity

A **prediction event** is one token the model has to predict: every word of every test story, plus the EOS that ends each story (BOS is given, never predicted). The four sentences of §2 have seventeen. Over M prediction events, the average log loss and the **perplexity** are

$$\widehat{L} = -\frac{1}{M} \sum_t \log q(x_t \mid x_{<t}), \qquad \mathrm{PPL} = e^{\widehat{L}}.$$

Perplexity is the exponentiated average loss, equivalently the inverse geometric mean of the probabilities given to the observed tokens. A model uniform over K tokens has perplexity K, hence the usual reading: a model with perplexity 28 is, on average, as uncertain as a uniform choice among 28 tokens. In §2, *the cat sleeps* EOS has probability $1/4$ over four events, so $\mathrm{PPL} = 4^{1/4} = \sqrt{2}$, a score on a training sentence that says nothing about generalization.

Perplexities compare only between models that predict the same tokens: splitting words into pieces increases M and lowers the per-token perplexity without improving the model, and mapping rare words to `<unk>` makes them easier to predict. The word models of this lecture share one tokenizer and vocabulary (§1), so their perplexities compare with each other but not with perplexities reported elsewhere.

Bits per character and compression

To compare models with different tokens, divide the total loss in bits by the number of characters (or bytes), which no tokenizer changes: **bits per character** (or bits per byte), the quantity Shannon

estimated. Arithmetic coding turns any model into a lossless compressor whose output for a text of probability $q(x)$ is within two bits of $-\log_2 q(x)$ (Witten, Neal, and Cleary, 1987), so a model with 1.3 bits per character compresses text to about 1.3 bits per character. The character models of this lecture are compared this way; a uniform guess over TinyStories' 34 symbols scores $\log_2 34 \approx 5.09$.

Training, validation, and test data

Parameters are fitted to the training text. **Hyperparameters**, such as the order n or the strength of smoothing (§4), cannot be, since on the training text the largest n and no smoothing always win; they are chosen on **validation** data, and the chosen model is scored once on **test** data. The split is by *document*: sentences of one story share names and phrases, so testing on sentences of training stories tests on text partly seen. (Web-scale corpora must also be deduplicated before splitting.)

Under this score the unsmoothed models of §2 fail outright: one test event never seen in training gets probability zero and makes the loss of the whole test set infinite, and 18% of test trigrams are of that kind.

4. Smoothing

Smoothing moves probability from the events seen in training to events that were not. Every method answers two questions: how much probability to set aside, and how to share it among the unseen events.

Add-α

The oldest method pretends every event was seen α more times than it was:

$$q_\alpha(w \mid h) = \frac{c(h, w) + \alpha}{c(h) + \alpha|\mathcal{V}|}.$$

With $\alpha = 1$ this is Laplace's rule of succession. On the four sentences of §2 it gives *a cat sleeps* EOS probability 1/900 instead of zero, at a price: *the cat sleeps* falls from 1/4 to 3/550, and the eight sentences of the form *the/a cat/dog sleeps/eats* together keep only 41/1980, about 2%, of the probability, against 3/4 without smoothing. With a large vocabulary, add-one gives the unseen events far too much: on TinyStories its trigram model is *worse* than its bigram model (test perplexities 421 and 110), because a trigram history is seen few times and the 8,681 pseudo-counts swamp the real ones. A small α chosen on validation data helps (51.7 for bigrams with $\alpha = 0.003$), but the pseudo-count is the same for an unseen word that is plausible in the context and one that is not.

The four sentences also show why α is chosen on validation data. Take the validation set {*a cat sleeps*, *a cat eats*} and the test set {*the dog eats*, *a dog sleeps*}:

α	**0**	**0.01**	**0.1**	**0.5**	**1**
validation loss (nats per token)	∞	1.795	**1.477**	1.623	1.751
test loss (nats per token)	0.607	0.635	0.842	1.285	1.516

Validation picks $\alpha = 0.1$, and its test loss, 0.842 nats per token, is the reported result. The test set would have picked $\alpha = 0$, because every transition in its two sentences happens to occur in training, and that model gives the validation sentences probability zero. Choosing on the test set reports the luck of the sample as skill.

Borrowing from shorter contexts

How much probability should the unseen events get in total? Alan Turing and I. J. Good, working on Enigma at Bletchley Park, answered: the fraction of training events seen exactly once, since a type seen once would have been unseen had that occurrence been held out. On TinyStories this predicts that 3.5% of the bigrams in new text are unseen; the test stories have 3.8% (§A.2).

How should it be shared? Through shorter contexts: if *the purple dog* was never seen, *purple dog* or *dog* may have been. **Interpolation** mixes the estimates of all orders; **backoff** uses the longest context that was seen. **Kneser–Ney smoothing** (Kneser and Ney, 1995), the best count-based method, adds

two refinements. It subtracts a fixed discount, about 0.75, from every count, roughly what Good and Turing's analysis says an observed count overstates. And for the shorter context it asks of a word not how often it occurs but after how many different words: *upon* occurs 10,921 times in TinyStories but after only 14 different words (nearly always *once*), *toy* 3,223 times after 200, so in a new context *toy* is the better guess. §A.3 gives the formulas.

On TinyStories, the test perplexities of the word models are:

Model	**unigram**	**bigram**	**trigram**	**4-gram**
Unsmoothed (maximum likelihood)	300	∞	∞	∞
Add-one	301	110	421	
Add-α, α tuned on validation	300	52	59	
Interpolated Kneser–Ney, $D = 0.75$		47	28	25

Smoothing is not a detail. With the same counts, the trigram model's perplexity ranges from infinite to 28 depending on how probability is shared out, and only a good smoother turns the extra context of a longer history into better predictions.

How much context?

Long character contexts are cheap to count, which makes the trade-off between context and data easy to see. Figure 4 shows character n-gram models of TinyStories with interpolated Kneser–Ney. The held-out loss falls from 4.2 bits per character for single-letter frequencies to 1.43 at $n = 6$, reaches its minimum, 1.26, at $n = 10$, and is slightly worse at $n = 11$. The training loss keeps falling, to 0.74 at $n = 11$, and without smoothing a growing share of held-out characters would get probability zero: 2% at $n = 6$, 14% at $n = 9$, 27% at $n = 11$. This is the overfitting of Lecture 1 (Section 3) in its simplest form: beyond some order the model describes the training text rather than the language. Smoothing pushes that order out but cannot remove it, and the table grows quickly: the 10-gram model stores 4.3 million counts.

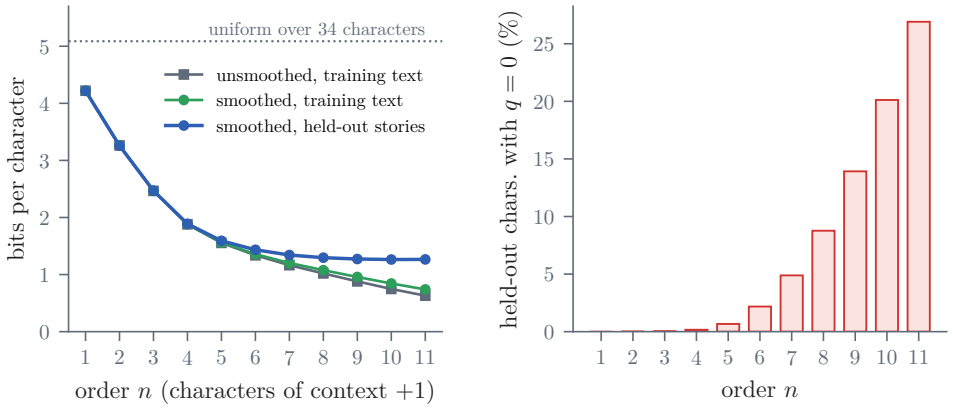


Figure 4. Character n-gram models of TinyStories (6,000 training stories, 500 held-out stories). Left: bits per character on training text and held-out text. Right: the fraction of held-out characters whose n-gram never occurs in the training stories, to which the unsmoothed model assigns probability zero.

What counting cannot do

Smoothing shares probability between a context and its shorter versions, not between *similar* contexts. What a trigram model learns about *the dog barked* tells it nothing about *the puppy barked*, because to a table of counts a word is an index, and two indices are either equal or unrelated. Using what is known about one word for another requires a representation in which words can be similar: the subject of the second part of the lecture.

5. Word embeddings

Why vectors

To a table of counts, a word is an index. Written as a vector, the index is **one-hot**: $\mathbf{x}_w \in \{0,1\}^{|\mathcal{V}|}$ with a single 1 in position w. Any two one-hot vectors are orthogonal and $\sqrt{2}$ apart, so *dog* is as different from *puppy* as from *banana*, and nothing learned about one word applies to another.

A **word embedding** maps each word to a dense vector $\mathbf{v}_w \in \mathbb{R}^d$, with d typically between 50 and 1,000, so that words used alike get nearby vectors (Figure 5). Nearness is usually measured by **cosine similarity**, $\cos(\mathbf{v}, \mathbf{v}') = \mathbf{v} \cdot \mathbf{v}' / (\|\mathbf{v}\| \|\mathbf{v}'\|)$, which ignores length (length mostly tracks frequency). Such vectors do three things indices cannot. Evidence transfers: a model that reads vectors treats *puppy* much like *dog* even where it has seen only *dog*. They are compact features that other models can reuse. And questions about meaning become questions about geometry, such as which words are nearest to a given one, which can be answered quickly at scale.

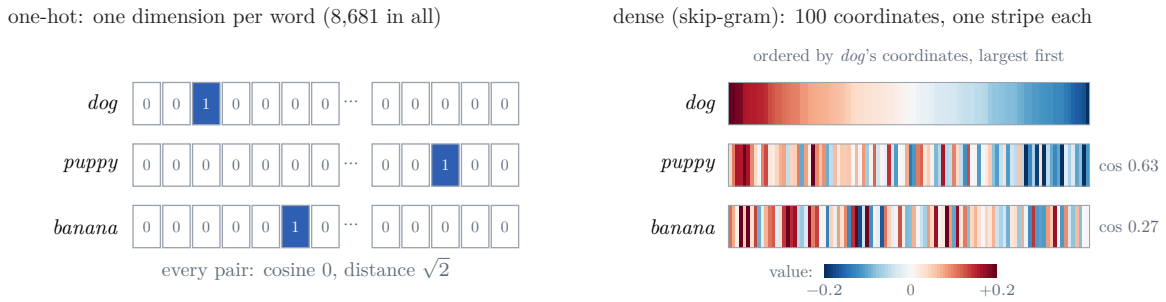


Figure 5. Two ways to represent a word as a vector. Left: one-hot vectors, one dimension per word of the vocabulary; all pairs are equally far apart. Right: the skip-gram vectors trained below, all 100 coordinates, one colored stripe per coordinate (red positive, blue negative, white near zero; each vector has length 1). The coordinates are put in the order of "dog"'s, from its largest to its smallest, so "dog"'s row runs from red to blue. "puppy" follows the same trend, red on the left and blue on the right (cosine 0.63); "banana" does not (0.27).

The idea behind them is Harris's distributional hypothesis from Lecture 1 (Section 6): words that occur in similar environments have similar meanings. A reader who meets *A bottle of tezgüino is on the table. Everybody likes tezgüino. Tezgüino makes you drunk. We make tezgüino out of corn.* knows what tezgüino is, an alcoholic drink made from corn, though no one defined it (Lin, 1998, after Nida, 1975). This section builds embeddings from it in two ways, by counting contexts and by predicting them, and shows that the two are closer than they look.

Where embeddings are used

The same recipe applies to anything that occurs in contexts: represent each object by a vector, and train the vectors so that objects occurring in similar contexts are near each other.

- **Search.** Matching words fails when a query and a document use different words for the same thing (*cheap flights* and *inexpensive airfare*); matching embeddings by nearness does not. An early system trained the embeddings on which results users clicked (Huang et al., 2013); **dense retrieval** embeds passages with a neural network and retrieves those nearest to the question (Karpukhin et al., 2020). Sentence embeddings, trained so that sentences with similar meanings are near each other (Reimers and Gurevych, 2019), are the usual building block.
- **Vector databases.** With millions or billions of vectors, the central operation is the nearest-neighbor query. Exact search compares the query with every stored vector, $O(Nd)$ work per query. Vector databases answer **approximate** nearest-neighbor queries far faster, by locality-sensitive hashing (a relative of the MinHash method of Week 5), by quantizing vectors into short codes (Johnson, Douze, and Jégou, 2017), or by walking a graph that links each vector to near neighbors (Malkov and Yashunin, 2016).
- **Recommendation.** Treat a user's sequence of clicks, songs, or viewed listings as a sentence and the items as words, and train skip-gram (below) on the sessions: items that appear in similar sessions get nearby vectors, and "users who looked at this also looked at" becomes a nearest-neighbor query (Grbovic and Cheng, 2018, for listings at Airbnb).

- **Language models.** Every neural language model begins with a table from tokens to vectors (§6), and many deployed assistants consult documents they were not trained on by retrieving passages by embedding similarity.

A short history

The geometry of meaning is older than the neural networks that made it popular. Charles Osgood's *The Measurement of Meaning* (1957) placed words as points in a space whose axes came from people's ratings (good–bad, strong–weak). Information retrieval represented documents as vectors of word counts from the 1960s on (Salton, Wong, and Yang, 1975), and **latent semantic analysis** (1990) compressed such count matrices with a singular value decomposition, giving dense word vectors from counts (Landauer and Dumais, 1997, give the theory). Vectors learned by networks appeared in parallel. Geoffrey Hinton's 1986 network, trained on facts about two family trees, learned vectors for people whose dimensions tracked generation and nationality without being told about either; Jeffrey Elman's recurrent network of 1990, trained to predict the next word, had hidden states that grouped nouns and verbs, and animate and inanimate nouns (Elman, 1990). The neural language model of Bengio et al. (2003) (§6) learned a word table as a by-product of next-word prediction, and Collobert and Weston (2008) trained vectors on Wikipedia with a cheap ranking objective and reused them across language tasks (Collobert et al., 2011).

In 2013 a group at Google led by Tomas Mikolov stripped the language model down to what the vectors need: no hidden layer and no softmax over the vocabulary (Mikolov et al., 2013a; 2013b). Their **word2vec** learned good vectors from 1.6 billion words in less than a day, and they released vectors for 3 million words and phrases trained on about 100 billion words of Google News. The demonstration that vector arithmetic captures relations (*king* $-$ *man* $+$ *woman* $\approx$ *queen*) made embeddings a standard tool, and the paper received the NeurIPS Test of Time Award in 2023. Within a year, analysis showed that word2vec is close to the counting methods it replaced (see below).

Vectors from counts

The counting route has three steps: count which words occur near which, reweight the counts, and compress each word's row of counts into a short vector (Figure 6).

Count. Take the 3,000 most frequent words of the training stories (punctuation removed) and a window of two positions on each side. Entry $n(w, c)$ of a $3{,}000 \times 3{,}000$ matrix is the number of times word c occurs within two positions of word w. Rows and columns are indexed by the same words, and the matrix is symmetric. The row of w summarizes the company w keeps. Following the literature, w is the **word** and c the **context word**; the names only distinguish the two roles. Figure 6a shows nine rows of the matrix and eight of its columns.

Reweight. Raw counts are dominated by frequent words: *the* occurs near everything, and *said* near every character who speaks. The **pointwise mutual information** compares the observed co-occurrence with what independence would predict:

$$\mathrm{PMI}(w, c) = \log \frac{p(w, c)}{p(w)\, p(c)}, \qquad p(w, c) = \frac{n(w, c)}{N}, \quad p(w) = \frac{\sum_{c'} n(w, c')}{N}, \quad p(c) = \frac{\sum_{w'} n(w', c)}{N},$$

where N is the sum of all the counts (Church and Hanks, 1990). PMI is positive when the pair occurs together more often than chance, zero for independence, and negative when the words avoid each other. In TinyStories, *ice* and *cream* have PMI 6.7, so they co-occur about $e^{6.7} \approx 800$ times more often than chance; *once* and *upon* 5.0; *the* and *dog* 1.2, a weak association despite 2,596 co-occurrences; *happy* and *sad* -2.2. Setting negative and undefined entries (pairs never seen together) to zero gives the **positive PMI** matrix M, which works better in practice (Bullinaria and Levy, 2007). In Figure 6b each group of words lights up its own contexts: the colors *color* and *paint*, *dog barked*, *cat meow*, *bird fly*, the family *hug* and *said*.

Compress. A row of M has 3,000 entries, 90% of them zero, and two words with related but different neighbors share few nonzero entries. A truncated singular value decomposition compresses the rows: write $M \approx U_d \Sigma_d V_d^\top$, keeping the d largest singular values, and give word w the vector $\mathbf{v}_w$, row w of $U_d \Sigma_d^{1/2}$, with d numbers instead of 3,000 (Figure 6c, $d = 100$). The compression keeps the directions in which the rows vary most, so words with similar, not necessarily identical, neighbors get similar vectors. This is latent semantic analysis (Deerwester et al., 1990) applied to word–context counts. By cosine similarity the nine words of Figure 6d fall into three blocks, colors, animals, and family. Over the whole vocabulary, the nearest neighbors are:

word	**nearest neighbors**
mom	dad, mommy, mother, parents, mum
red	blue, yellow, green, pink, purple
ran	went, walked, headed, jumped, skipped
sad	scared, upset, ashamed, embarrassed, lonely
apple	berry, fruit, melon, cherry, cookie
three	years, five, counted, four, two

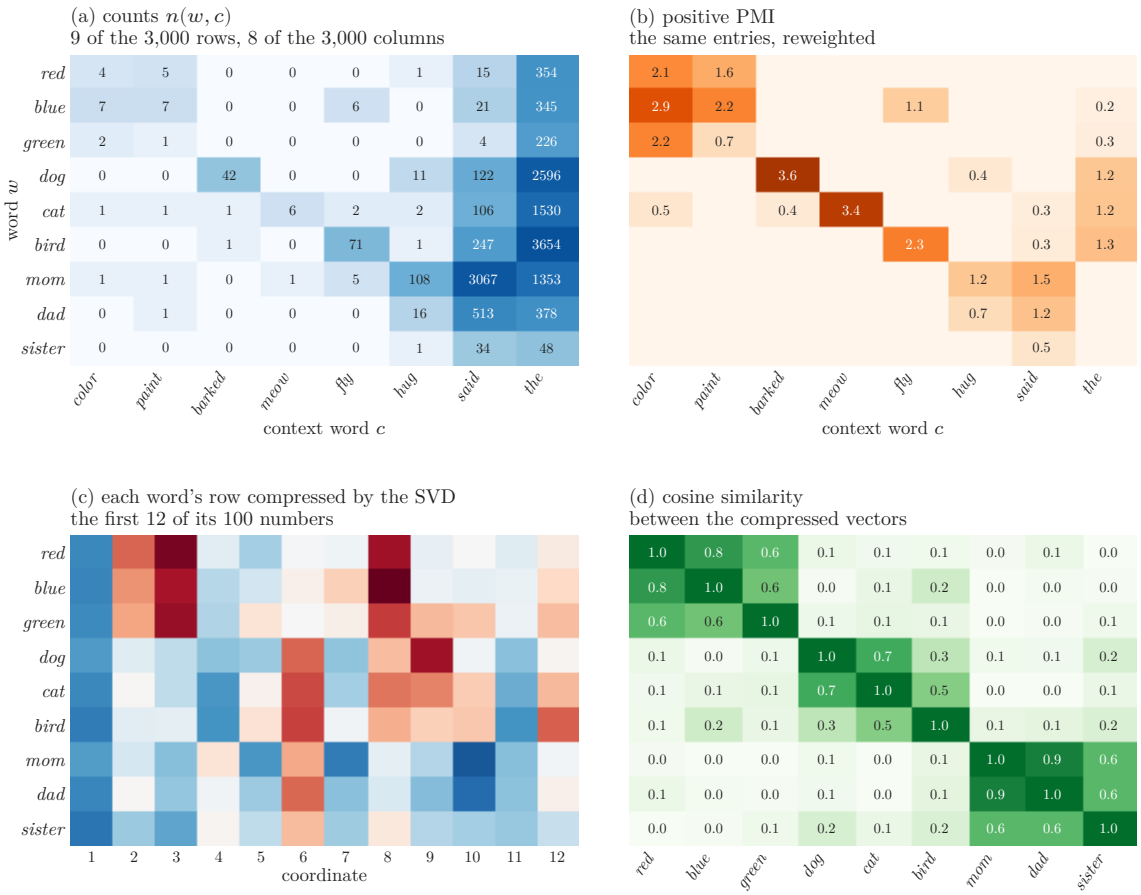


Figure 6. From counts to vectors, for nine words of TinyStories. (a) Part of the 3,000 × 3,000 count matrix: nine of its rows (the words w) and eight of its columns (context words c); each entry counts how often c occurs within two positions of w. (b) The positive PMI of the same entries, which discounts frequent contexts such as "the" and "said". (c) After a truncated SVD of the full positive PMI matrix, each word is a vector of 100 numbers; the first 12 are shown (blue negative, red positive). (d) The cosine similarity between the nine words' vectors.

No one told the method what a color, a parent, or a number is. The groups come from the company the words keep, as Harris predicted; the same method applied to two Austen novels produced the word-vector figure of Lecture 1 (Section 6).

Vectors from prediction: word2vec

word2vec is two papers and a program from Google (Mikolov et al., 2013a; 2013b). The first proposed two prediction models stripped down to the vectors, CBOW (predict a word from the average of its neighbors' vectors) and skip-gram (predict each neighbor from the word), and showed that the vectors answer analogy questions by arithmetic; the second made skip-gram faster and better with negative sampling and the subsampling of frequent words, and learned vectors for phrases such as *New York*. The program and vectors trained on Google News were released freely. This section follows skip-gram with negative sampling.

word2vec starts from the same raw material as the count matrix, the pairs of words that occur near each other, but instead of counting the pairs it trains vectors to predict them.

The data. At each position of the training text, the word there is paired with each word within two positions of it (Figure 7a): every pair for which the count matrix adds one is a training example (w, c).

The parameters. Two learned tables of vectors (Figure 7b): a **word vector** $\mathbf{v}_w \in \mathbb{R}^d$ for every word, used when the word is at the center of a window, and a **context vector** $\mathbf{u}_c \in \mathbb{R}^d$, used when it is the neighbor. The score of a pair is the dot product $\mathbf{v}_w \cdot \mathbf{u}_c$. As in word2vec, the word vectors start small and random and the context vectors at zero. (Two tables rather than one, because a word is a poor predictor of itself as a neighbor, and a single table would push $\mathbf{v}_w \cdot \mathbf{v}_w$ to be small. After training, the word vectors are kept.)

The skip-gram model. In its first form, the scores predict the neighbor from the word with a softmax over the vocabulary,

$$q(c \mid w) = \frac{\exp(\mathbf{v}_w \cdot \mathbf{u}_c)}{\sum_{c' \in \mathcal{V}} \exp(\mathbf{v}_w \cdot \mathbf{u}_{c'})},$$

and training maximizes the log likelihood $\sum \log q(c \mid w)$ over all the pairs: a classifier with one class per word, much like the bigram network of §6, except that it predicts neighbors on both sides instead of the next word. (Its mirror image, **CBOW**, predicts a word from the average of its neighbors' vectors.) The cost is in the denominator: every pair needs a dot product with every context vector, $O(|\mathcal{V}|d)$ operations.

Negative sampling. word2vec replaces the prediction by a yes-or-no question: is this pair real, or made up? For each real pair (w, c), it draws k random words $c'_1, \ldots, c'_k$ from a **noise distribution** p_n, the word frequencies raised to the power 3/4, to make k made-up pairs (w, c'_i). It reads $\sigma(\mathbf{v}_w \cdot \mathbf{u}_c)$, with $\sigma(s) = 1/(1 + e^{-s})$, as the probability that a pair is real, and maximizes, for each real pair,

$$\log \sigma(\mathbf{v}_w \cdot \mathbf{u}_c) + \sum_{i=1}^{k} \log \sigma(-\mathbf{v}_w \cdot \mathbf{u}_{c'_i}),$$

the log likelihood of labeling the real pair 1 and the made-up pairs 0. Each term is a logistic regression, the model of Lecture 1's hands-on, with the features learned too, and the cost per pair is $O(kd)$ instead of $O(|\mathcal{V}|d)$. Negative sampling is a simplified form of noise-contrastive estimation (Gutmann and Hyvärinen, 2010).

Why this gives good vectors. A gradient step on a real pair moves $\mathbf{v}_w$ toward $\mathbf{u}_c$, and a step on a made-up pair moves it away from $\mathbf{u}_{c'}$. Two words that occur among the same neighbors are pulled toward the same context vectors, so their word vectors end up close: the distributional hypothesis, carried out by gradient descent.

Training is stochastic gradient descent on minibatches of pairs, with two smaller choices that matter in practice: very frequent words are randomly dropped from the text before pairs are formed, so that pairs with *the* do not dominate, and the window size is sampled, so that nearer words count more.

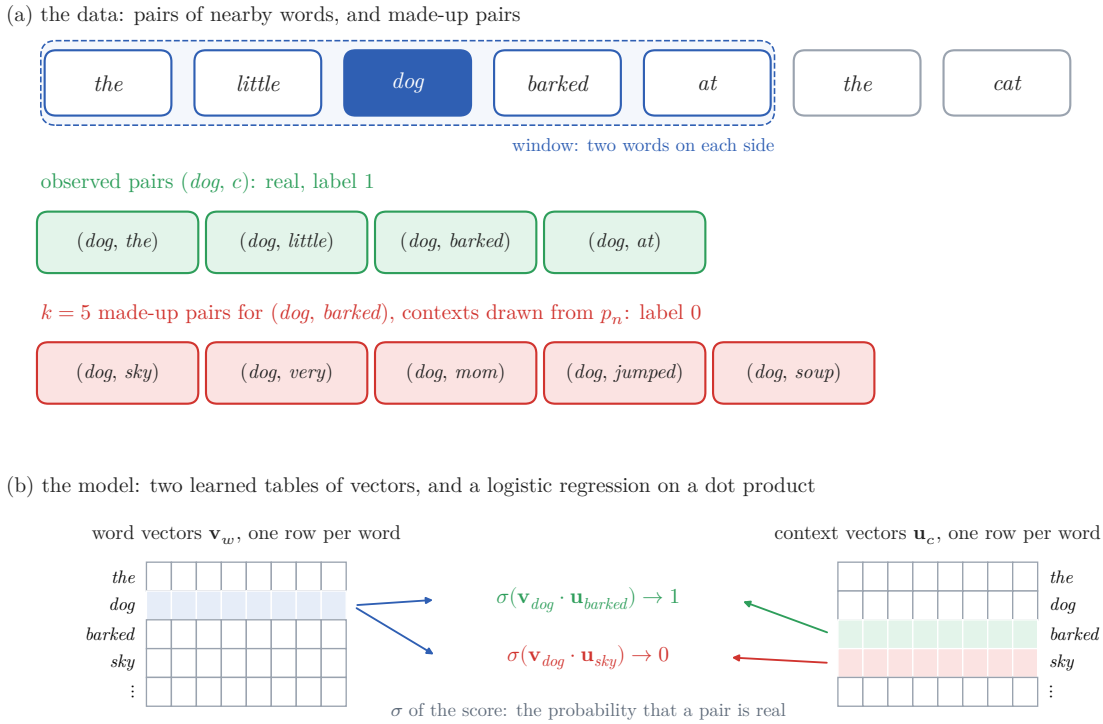


Figure 7. Skip-gram with negative sampling. (a) The training examples one position of a sentence produces: each word within the window around "dog" forms a real pair, and each real pair is joined by k made-up pairs whose context is drawn at random from p_n. (b) The model: a table of word vectors and a table of context vectors, both learned; the score of a pair is the dot product of the word's vector and the context's vector, and its sigmoid is the probability that the pair is real. Training pushes it toward 1 for real pairs and toward 0 for made-up ones.

On TinyStories (Part 4 of the notebook), each of the 6,453 words that occur at least five times in the training stories starts with a small random 100-dimensional word vector and a zero context vector. Training uses the pairs within two words of each other, about 2.7 million per pass after frequent words are thinned out, with $k = 5$ negative contexts per pair; ten passes take a few minutes on a laptop CPU. Figure 8 projects the vectors of 36 words from six groups to two dimensions, before and after training. Before training the groups are scattered. After training the colors, animals, family members, feelings, foods, and places each form a cluster, although nothing in the objective names a group: the only signal was which words occur near which. Within the feelings, *happy* and *excited* sit apart from *sad*, *angry*, *scared*, and *worried*. The nearest neighbors of *happy* are *grateful*, *thankful*, *thanked*, and *glad*; of *sad*, *upset*, *unhappy*, *heartbroken*, and *ashamed*; of *king*, *queen*, *prince*, *kingdom*, and *princess*; of *cake*, *pizza*, *cookies*, *oven*, and *baked*.

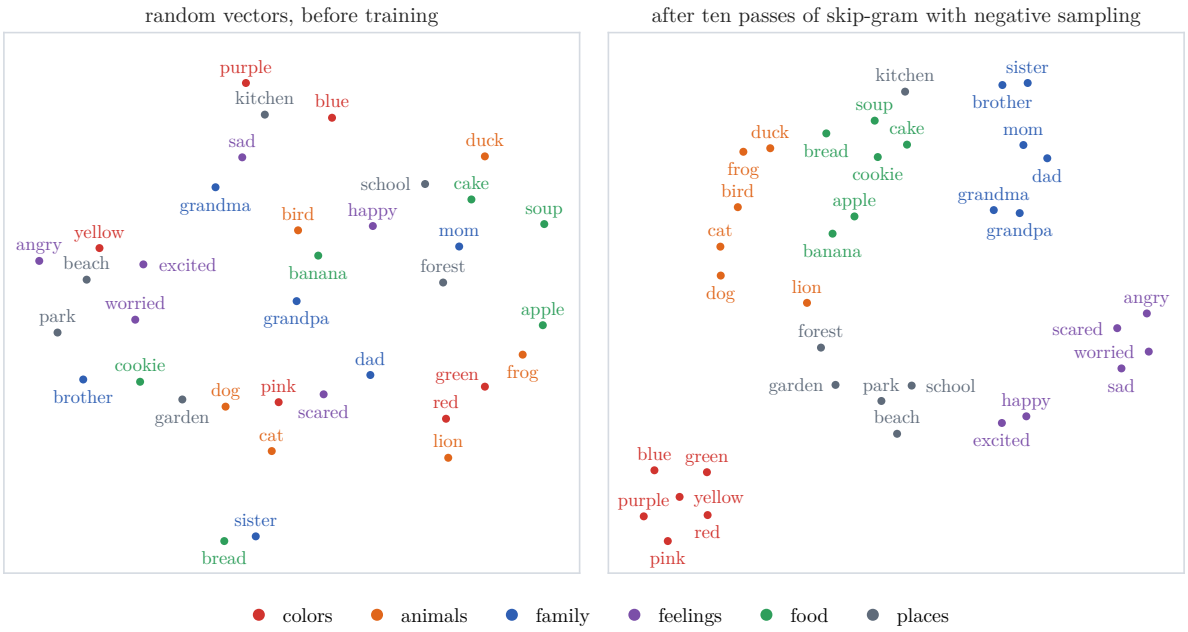


Figure 8. Word vectors of TinyStories projected to two dimensions. Left: the random vectors skip-gram training starts from. Right: the same words after training with negative sampling. Colors mark the six groups, which the training never saw. The projection is t-SNE on cosine distance, which keeps near neighbors near; distances between clusters in the picture mean little.

Relations between words can appear as directions. The vector $\mathbf{v}_{\text{girl}} - \mathbf{v}_{\text{boy}} + \mathbf{v}_{\text{he}}$ is closest to *she*, and the same arithmetic gives *his* → *her*, *mommy* → *daddy* (from *mom* → *dad*), and *walk* → *walked* (from *run* → *ran*). Figure 9 shows why: the offsets from *boy* to *girl*, *he* to *she*, and *his* to *her* are nearly parallel and of nearly equal length. On a corpus this small the picture is only partly true: the offset from *man* to *woman* is shorter and tilted, *brother* → *sister* comes first only barely (cosine 0.56, tied with *mandy's*), *prince* → *princess* misses the top three, and *three* goes to *four* only in third place. Relations a corpus repeats thousands of times become consistent directions; rarer ones do not, and whether they come out right changes from one training run to the next.

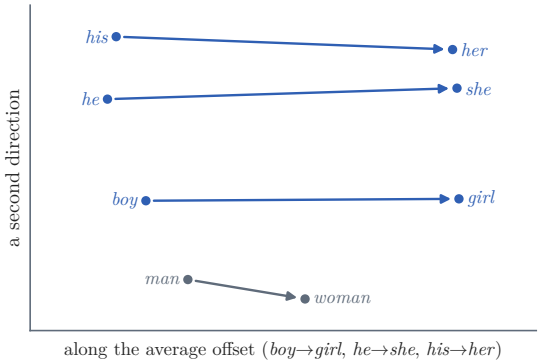


Figure 9. Relations as directions. Skip-gram vectors of four pairs of words, projected onto the average of the offsets "boy" to "girl", "he" to "she", and "his" to "her" (horizontal) and one further direction (vertical). The three frequent pairs have nearly parallel offsets of similar length; the offset from "man" to "woman" is shorter.

Counting and prediction meet

Is prediction a different principle from counting? Levy and Goldberg (2014) showed that it is not: the dot products that skip-gram learns are, at best, the PMI values of the count matrix. The argument uses the notation of the counting route: $n(w, c)$ is the number of times the pair (w, c) occurs in the training text, $n(w) = \sum_c n(w, c)$ and $n(c) = \sum_w n(w, c)$ are the numbers of pairs with w as the word and with c as the context, and N is the total number of pairs. It takes three steps.

Collect the terms of one pair. Fix a word w and a context c, and write $s = \mathbf{v}_w \cdot \mathbf{u}_c$. Summed over the training text, the objective contains $\log \sigma(s)$ once for every time (w, c) occurs as a real pair,

$n(w, c)$ times. It also contains $\log \sigma(-s)$ for every time (w, c) is made up: each of the $n(w)$ real pairs with w as the word brings k made-up contexts, each drawn from the noise distribution p_n, so c is drawn $k\, n(w)\, p_n(c)$ times on average. The part of the objective that depends on s is therefore

$$\ell(s) = n(w, c) \log \sigma(s) + k\, n(w)\, p_n(c) \log \sigma(-s) \,:$$

the first term pulls s up, the second down, weighted by how often the pair is real and how often it is made up.

Maximize. If the vectors had enough dimensions for every score s to be set independently, each pair's $\ell(s)$ would be maximized on its own, where its derivative vanishes. Since $\frac{d}{ds} \log \sigma(s) = \sigma(-s)$ and $\frac{d}{ds} \log \sigma(-s) = -\sigma(s)$, the condition is $n(w, c)\, \sigma(-s) = k\, n(w)\, p_n(c)\, \sigma(s)$, and since $\sigma(s)/\sigma(-s) = e^s$,

$$s = \log \frac{n(w, c)}{k\, n(w)\, p_n(c)}.$$

Read off the PMI. With $p_n(c) = n(c)/N$, the plain context frequency,

$$\mathbf{v}_w \cdot \mathbf{u}_c = \log \frac{n(w, c)\, N}{n(w)\, n(c)} - \log k = \mathrm{PMI}(w, c) - \log k,$$

because $n(w, c)/N$, $n(w)/N$, and $n(c)/N$ are the probabilities $p(w, c)$, $p(w)$, $p(c)$ in the PMI. With word2vec's 3/4 power, the same steps give the PMI computed with the smoothed context frequencies (§A.5).

So skip-gram with negative sampling implicitly factorizes the PMI matrix shifted by $\log k$, which the counting route factorizes explicitly with the SVD (its positive part, unshifted). With 100 dimensions the scores cannot all be set independently, so both factorizations are approximate, and the two methods spread the error differently, but the target is the same.

The prediction can be checked on the TinyStories run. For the 6,757 word–context pairs that occur at least 20 times among the training pairs, Figure 10 plots the learned dot product $\mathbf{v}_w \cdot \mathbf{u}_c$ against the value computed from the counts. The correlation is 0.96 and the best-fitting line has slope 0.92: trained by stochastic gradient descent on a prediction task, the network has stored the PMI of every frequent pair in the dot products of its vectors.

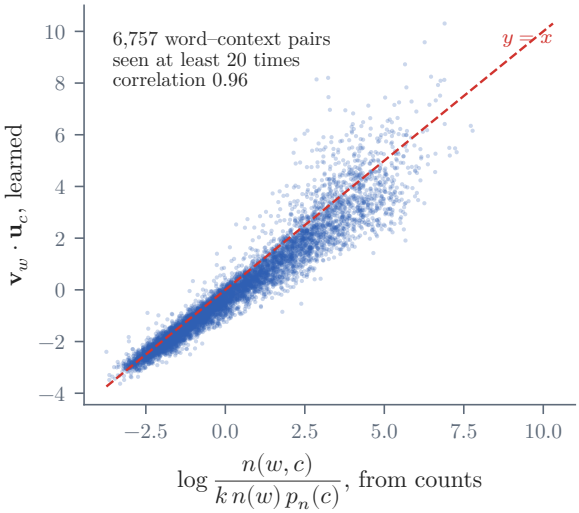


Figure 10. What skip-gram learned, against what counting predicts. Each point is a word–context pair from the TinyStories run that occurs at least 20 times. Horizontal: the shifted PMI that the argument above predicts, computed from the pair counts. Vertical: the dot product $\mathbf{v}_w \cdot \mathbf{u}_c$ of the trained vectors. Points on the dashed diagonal are pairs where the vectors learned exactly what the counts predict; the correlation is 0.96.

Levy, Goldberg, and Dagan (2015) found that with matched preprocessing (window, frequency subsampling, the 3/4 smoothing of context frequencies) count-based vectors perform about as well as word2vec's. **GloVe** (Pennington, Socher, and Manning, 2014) makes the connection explicit: it fits $\mathbf{v}_w \cdot \mathbf{u}_c + b_w + b_c \approx \log n(w, c)$ by weighted least squares over the nonzero counts. Prediction keeps two advantages: it streams through text without building the count matrix, and it generalizes, since a language model, which predicts the next word from many words at once, learns its word vectors the same way (§6).

What similarity captures, and what it misses

Distributional similarity is not synonymy. In TinyStories, *hot* and *cold* have cosine similarity 0.64, more than *happy* and *glad* (0.38); *small* is among the nearest neighbors of *big*, and *down* of *up*. Antonyms keep almost the same company: one is happy or sad *about* the same things. Similarity also mixes kinds of relatedness: the neighbors of *dog* include *cat*, but also *barks* and *tail*.

Embeddings also absorb the associations of their training text, including social stereotypes. An analogy query $a : b :: c : ?$ is answered with the vectors alone, as in Figure 9: compute $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$ and return the word whose vector is nearest to it by cosine, leaving out a, b, and c themselves. No language model is involved, only the word vectors and a nearest-neighbor search. In the word2vec vectors trained on Google News, *man* : *computer programmer* :: *woman* : ? returns *homemaker* (Bolukbasi et al., 2016). Part of this comes from how the query is answered: the vector nearest to the target is *computer programmer* itself, and *homemaker* is second; the usual rule excludes the query words, so *homemaker* is what gets reported (Nissim, van Noord, and van der Goot, 2020). Still, *homemaker* is the runner-up: the association is in the vectors either way. A system that uses such vectors for search or hiring inherits these associations unless they are measured and corrected.

Finally, there is one vector per word type, so *bank* gets a single vector that blends the river bank and the bank that lends money. Rare and unseen words get poor vectors or none; building vectors from pieces of words helps (Bojanowski et al., 2017). The deeper fix is to compute a vector for each *occurrence* of a word from the words around it (Peters et al., 2018), which is what the Transformer does (Week 4).

Word vectors know which words are alike but predict nothing; n-gram models predict but do not know which words are alike. The third part of the lecture builds models that do both.

6. Neural language models

The bigram model as a network

The first step changes how a model is computed, not what it is. It follows the first of Karpathy's *makemore* lectures, *The spelled-out intro to language modeling*, which does the same for the letters of names.

A character bigram model is a table with one row of probabilities $q(\cdot \mid a)$ for each previous character a. Write the row as the output of a network (Figure 11, left): represent a by its one-hot vector $\mathbf{x}_a$; multiply by a weight matrix W, with one row and one column per character, to get a score for every next character, $\mathbf{z} = W^\top \mathbf{x}_a$, which is row a of W; and turn the scores into probabilities with the **softmax**:

$$q(b \mid a) = \frac{\exp(W_{ab})}{\sum_{b'} \exp(W_{ab'})}.$$

The scores are called **logits**. The softmax makes each row positive and sum to one, and any row of positive probabilities can be produced this way (set $W_{ab} = \log q(b \mid a)$), so the network represents the bigram models and nothing else.

Train W as in Lecture 1's hands-on. Start from $W = 0$, which makes every row uniform (5.09 bits per character), and repeat: draw a minibatch of 4,096 (previous, next) character pairs from the training stories, compute the average log loss of the next characters, backpropagate, and take a gradient step. Each row's loss depends only on that row, and §2 found its minimum: the relative frequencies. So gradient descent must arrive at the count table, and it does. After 3,000 steps, a few seconds of computation, the network's loss is 3.262 bits per character on the training stories and 3.258 on held-out stories, the same as the count model's, and the two tables are indistinguishable (Figure 11, middle and right). Maximum likelihood gives one answer, whether computed by counting or by gradient descent.

Smoothing has a counterpart. A penalty $\lambda \sum_{a,b} W_{ab}^2$ added to the loss pulls every row toward $W = 0$, the uniform distribution, and the pull wins in rows supported by little data, as add-α's pseudo-counts do. Penalties of this kind are called **regularization**. (Characters barely need it: 778 of the 1,156 possible pairs occur in training, and only 4 of the 429,104 held-out pairs do not.)

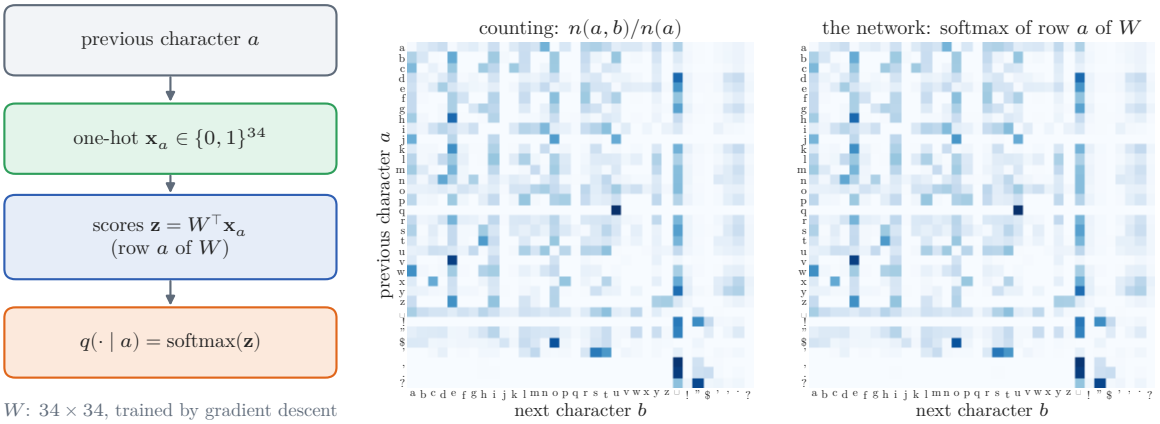


Figure 11. The bigram model as a neural network. Left: the network, a single weight matrix followed by a softmax. Middle: the character bigram probabilities of TinyStories, by counting. Right: the softmax of the rows of W after training by gradient descent on the log loss. Darker cells are more probable next characters (square-root scale); the two tables agree.

So far nothing is gained: the network is a slower way to fill in the same table. But the table is now the output of a differentiable function, trained by gradient descent, and the function can be changed. Two changes lead to the neural language models of this section.

From a table to word vectors

The first change matters for words. A word bigram network's matrix W has a row for each previous word (8,682, with BOS) and a column for each next word (8,681), about 75 million numbers, and each row is learned only from the occurrences of its own word, like a row of counts. Nothing is shared. Factor the matrix through a small space: map the previous word v to a vector $C(v) \in \mathbb{R}^m$, and the vector to one score per next word,

$$q(\cdot \mid v) = \mathrm{softmax}(\mathbf{b} + U\, C(v)).$$

The table C holds a row of m numbers per word, and the matrix U turns a vector into scores. This is still a bigram model, but its matrix of scores, $CU^\top$, has rank at most m, and that forces sharing: two words followed by similar words must get similar vectors, and a rare word borrows the predictions of the words near it. The rows of C are word embeddings, learned for prediction as word2vec's are (§5). With $m = 64$ the network has 1.1 million parameters instead of 75 million. Trained in the same way (Part 5 of the notebook), it reaches a test perplexity of 49.9, as good as the smoothed count bigrams of §4 (51.7 with tuned add-α, 47.5 with Kneser–Ney), with no smoothing rule written by hand.

The second change gives the network more than one word of context.

The fixed-window model

Bengio, Ducharme, Vincent, and Jauvin (2003) made both changes, a decade before word2vec. Their **neural probabilistic language model** keeps the n-gram assumption, that the next word depends only on the previous $n-1$, but computes the conditional distribution with a neural network (Figure 12). One table, shared by all positions, maps each word x to a learned vector $C(x) \in \mathbb{R}^m$. The $n-1$ vectors of the history are concatenated into $\mathbf{e}$, passed through a hidden layer, and turned into a distribution over the vocabulary by a softmax:

$$\mathbf{h} = \tanh(\mathbf{d} + H\mathbf{e}), \qquad q(\cdot \mid x_{t-n+1:t-1}) = \mathrm{softmax}(\mathbf{b} + U\mathbf{h}).$$

Each of the h coordinates of $\mathbf{h}$ is a **hidden unit**, the tanh of a weighted sum of the inputs: a neuron in the sense of Lecture 1. (Most of the original experiments also had direct connections from $\mathbf{e}$ to the output.) With $n = 2$ and no hidden layer this is the factored bigram network; the hidden layer lets the model combine the words of the history rather than add up their separate effects. Karpathy's *Building makemore Part 2: MLP* builds the same model for characters.

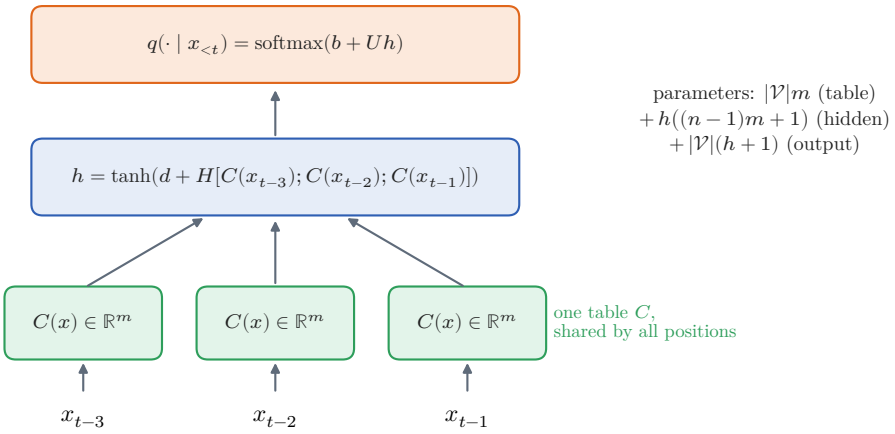


Figure 12. The neural probabilistic language model of Bengio et al. (2003), here with a three-word history. One table C maps every word to a vector; the vectors of the history feed a hidden layer and a softmax over the vocabulary.

Two things change relative to an n-gram model. First, the number of parameters no longer grows with the number of possible histories: with vocabulary size $|\mathcal{V}|$, embedding size m, and h hidden units, there are $|\mathcal{V}|m$ parameters in the table, $h((n-1)m+1)$ in the hidden layer, and $|\mathcal{V}|(h+1)$ in the output layer, linear in n instead of exponential. Second, similar histories get similar predictions automatically: if *dog* and *puppy* have nearby vectors, histories containing them produce nearby hidden states and distributions, so evidence about one transfers to the other. This is the sharing of statistical strength that Pereira's class-based model (Lecture 1, Section 6) achieved with word classes, now with a vector per word.

Training and generation

The training data are windows (Figure 13, top): every position of every training story gives one example, the four previous words (padded with BOS at the start of a story) and the word that follows, 3.56 million examples in all. Training repeats one step: draw a random minibatch of 512 examples, compute the model's distribution for each, average the log loss of the words that actually follow, backpropagate through the softmax, the hidden layer, and the rows of C that were looked up, and update every parameter with Adam, a form of gradient descent with a separate, adaptive step size for each parameter (Week 5). The word vectors start random and are trained by the same gradients as everything else: a row of C moves only when its word appears in a window, in whatever direction helps predict the words that follow it.

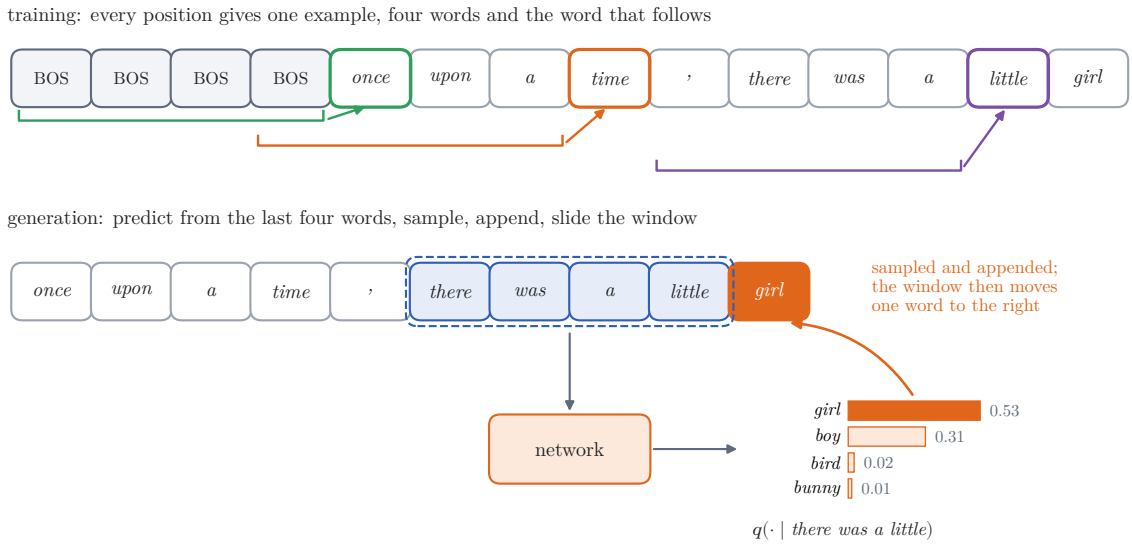


Figure 13. The fixed-window model in training and in generation. Top: every position of a story gives one training example, the four preceding words (BOS at the start) and the word that follows; three of them are marked. Bottom: to generate, the model reads the last four words, computes a distribution over the next word (here its actual top four after "there was a little"), samples one, appends it, and moves the window.

The model of the notes has a four-word history, $m = 64$, and $h = 128$: 1.7 million parameters. In Part 5 of the notebook its validation perplexity is 155 after 100 steps and 65 after 500, about 45 seconds of training. Two full passes over the training stories, about 14,000 steps and ten minutes on two CPU cores, bring it to 39 after 2,000 steps, 31 after 6,000, and 27.2 at the end; on the test stories it scores 26.9.

The price is computation: every prediction needs a product with the $|\mathcal{V}| \times h$ output matrix and a softmax over the whole vocabulary, where an n-gram model looks up a table entry, and 1.1 of the model's 1.7 million parameters are in the output layer. Bengio et al. (2003) trained their news-corpus models for five passes over the data, about three weeks on 40 CPUs.

Generation runs the model forward and feeds its own output back (Figure 13, bottom). Start with four BOS symbols; compute $q(\cdot \mid \text{BOS}, \text{BOS}, \text{BOS}, \text{BOS})$, which puts 0.68 on *once*; sample a word, append it, move the window one word to the right, and repeat until EOS is sampled. A **temperature** T makes samples more or less adventurous: dividing the logits by T before the softmax raises each probability to the power $1/T$ and renormalizes, so $T < 1$ sharpens the distribution toward its most probable words, $T > 1$ flattens it, and $T \to 0$ always picks the most probable word.

Results

On the Brown corpus (about a million words) Bengio et al. (2003) reported a test perplexity of 252 for their best model (a neural model averaged with a trigram model), against 312 for the best n-gram model they compared, a class-based backoff model; on a 14-million-word news corpus, 109 against 117 for Kneser–Ney. On TinyStories the word models of this lecture compare as follows:

Model	Context	Parameters	Test perplexity
Kneser–Ney bigram	one word	counts	47.5
Factored bigram network, $m = 64$	one word	1.1 million	49.9
Kneser–Ney trigram	two words	counts	27.9
Fixed-window network	four words	1.7 million	26.9
Kneser–Ney 4-gram	three words	counts	24.6
Fixed-window network and 4-gram, averaged			21.4

The fixed-window network lands between the Kneser–Ney trigram and 4-gram models of §4. The two kinds of model make different errors: averaging the neural model's probabilities with the 4-gram model's, with weight 0.4 on the neural model chosen on validation data, gives 21.4, better than either, as Bengio et al. (2003) also found. Neural models pulled clearly ahead once they were larger, trained longer, and given longer contexts, as §8 shows on TinyStories.

What the model has learned

A trained model can be examined in three ways, each a few lines of code in Part 5 of the notebook.

Its next-word distributions (Figure 14, left). After *she was very* the TinyStories model predicts *excited* 0.16, *happy* 0.10, *sad* 0.06, *proud* 0.04, *curious* 0.04: a distribution over feelings, as a person would guess. After *they went to the* it predicts places, led by *park* 0.23 and *kitchen* 0.12. After *once upon a* it gives *time* probability 0.997, and after *the dog wagged its*, *tail* 0.96.

Its samples. At temperature 0.8:

> once upon a time, there was a little boy named timmy. timmy loved to play. from that day on, lily thought it was so cool. he looked around and saw a tiny mouse. he was so relieved. it was so heavy! he had to go home.

Each four-word stretch is fluent; the story is not.

Its word table C (Figure 14, right). The model was never asked about word meanings, yet its word vectors are grouped like word2vec's in Figure 8: colors with colors, family members with family members, places with places. Words that predict the same continuations are best represented alike, so the table learns the distributional similarity of §5 as a by-product of predicting the next word.

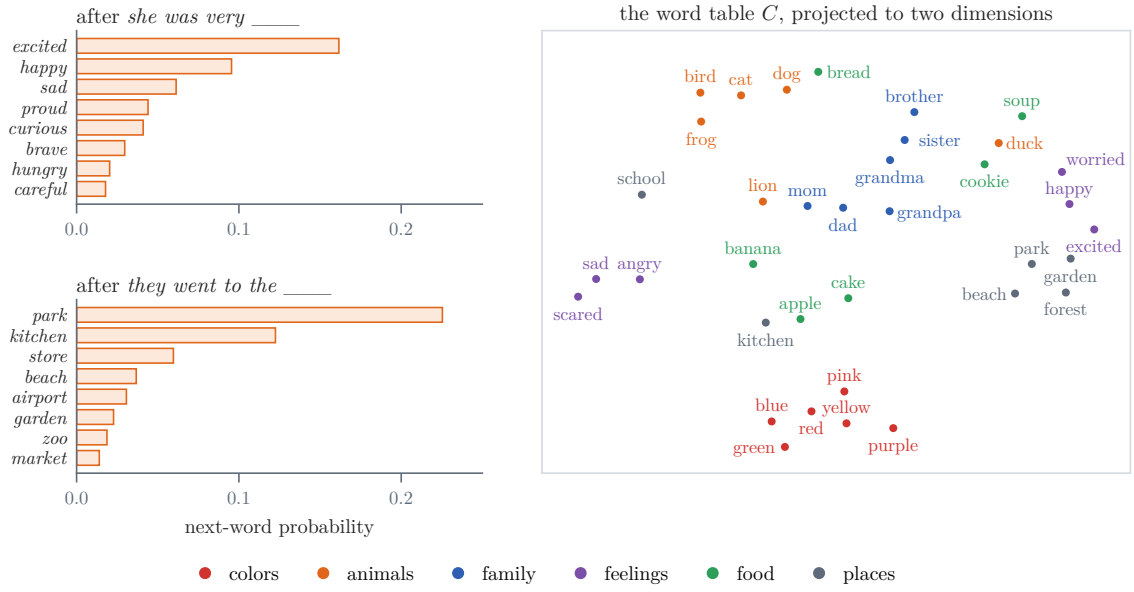


Figure 14. Inside the fixed-window language model of TinyStories. Left: its most probable next words after two contexts. Right: the rows of its word table C for the 36 words of Figure 8, projected to two dimensions in the same way. The table was trained only to predict the next word.

What the window cannot see

The sample shows the first limitation. The model sees four words, so *timmy* is forgotten by the time *lily* is chosen; a story carries its characters, objects, and events across hundreds of words, and no fixed window is long enough.

Widening the window is costly in a telling way. Each extra position adds hm weights to the hidden layer, and the weights for different positions are separate: what the model learns about *dog* one word back says nothing about *dog* three words back, which must be learned again from other examples. A window of 50 words would hold 50 separate copies of that knowledge, each trained on a fiftieth of the evidence: the n-gram model's failure to share, moved from words to positions.

Both problems have the same fix: apply the *same* weights at every position, and carry what has been read so far in a state updated one token at a time. That is a recurrent network.

7. Recurrent neural networks

The recurrence

A **recurrent neural network** (RNN) removes the window. It keeps a hidden state $\mathbf{h}_t \in \mathbb{R}^h$, a vector that summarizes the prefix, and updates it once per token with the same parameters at every step (Elman, 1990):

$$\mathbf{h}_t = \tanh(W\mathbf{h}_{t-1} + V\mathbf{e}_t + \mathbf{d}), \qquad q(\cdot \mid x_{\le t}) = \mathrm{softmax}(\mathbf{b} + U\mathbf{h}_t),$$

where $\mathbf{e}_t = C(x_t)$ is the vector of the current token and $\mathbf{h}_0$ is a fixed starting state. As in §6, each of the h coordinates of the state is a hidden unit: unit i holds the tanh of the i-th entry of $W\mathbf{h}_{t-1} + V\mathbf{e}_t + \mathbf{d}$, a number between -1 and 1. As a program, the model is a loop:

$\mathbf{h} \leftarrow \mathbf{h}_0$	the state: h numbers
for each token x of the text:	
$\quad \mathbf{h} \leftarrow \tanh(W\mathbf{h} + V\,C(x) + \mathbf{d})$	read the token into the state
$\quad q \leftarrow \mathrm{softmax}(\mathbf{b} + U\mathbf{h})$	the distribution of the next token

Figure 15 draws the model both ways. Rolled up, it is one cell whose state feeds back into itself through W. Unrolled over a sequence, the loop is a deep feed-forward network with one layer per token,

all layers sharing W, V, and $\mathbf{d}$. The number of parameters does not depend on the length of the context, what is learned at one position applies at every position, and in principle the state can carry information from arbitrarily far back.

The state plays the role of the hidden state of a hidden Markov model (§A.6), with two differences that matter: it is a vector of real numbers rather than one of K symbols, so h units can hold far more than $\log_2 K$ bits about the past, and it is a learned function of the previous state and the input, so similar prefixes lead to similar states. Mikolov et al. (2010) showed that recurrent language models beat well-tuned n-gram models in speech recognition, and Karpathy's essay (2015) showed recurrent networks trained on single characters writing passable Shakespeare, Wikipedia markup, LaTeX, and C code.

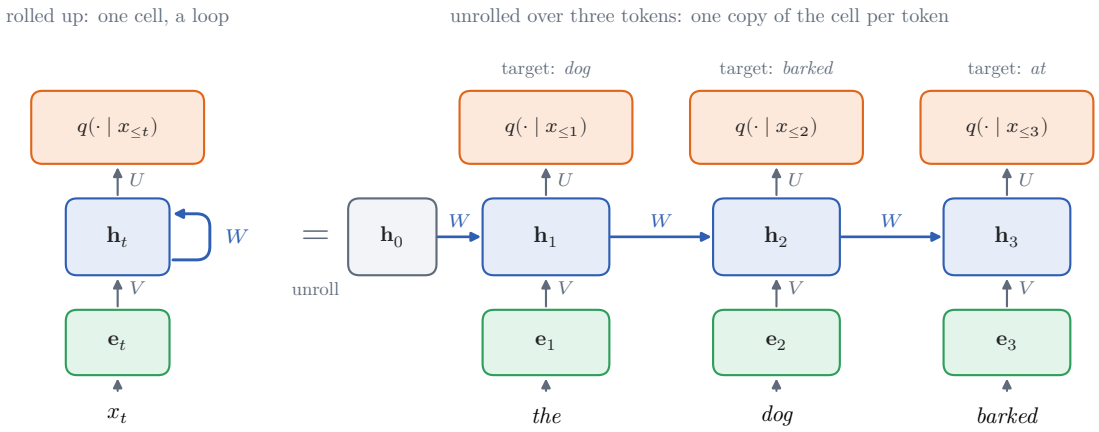


Figure 15. A recurrent language model. Left: rolled up, as the loop computes it, one cell whose state returns to it through W at the next step. Right: unrolled over three tokens, one copy of the cell per token. The same weights (W between states, V from inputs, U to outputs) are used at every step; the state $\mathbf{h}_t$ is the model's whole memory of the prefix.

One character at a time

Following Karpathy's essay, the recurrent models of these notes read and write characters. The vocabulary is tiny, 33 characters and the end-of-story symbol, so the softmax is cheap and the network's capacity goes into the recurrence. The stories are read as one long stream, each followed by the end-of-story symbol, so this one symbol does the work of both EOS and BOS: predicted, it ends a story; read as input, it is the context of the first character of the next. Nothing about words is given: spelling, word boundaries, and punctuation must be learned from the stream. And bits per character compare directly with the character n-grams of Figure 4. (The word-level version is in §8.)

Figure 16 shows a trained recurrent network reading a phrase. At each step it reads one character, updates its state, and outputs a distribution over the next character. At the start of a story it expects *o* (0.60, for *once*) and gives *t* 0.17. Within a word the choices narrow quickly: after *th* it puts 0.80 on *e*, after *the* 0.66 on a space (and 0.24 on *y*, for *they*), after *bar* 0.77 on *k*. The uncertainty is largest at the start of a word, where the network must guess what the story does next: after *the* it guesses *d* first, with only 0.19; after *the dog* it expects *w* (*was*, *went*), and *barked* is a surprise, with *b* at 0.03. Over the phrase it pays 1.73 bits per character, more than its held-out average of 1.47, because the first words of a story are the hardest to predict.

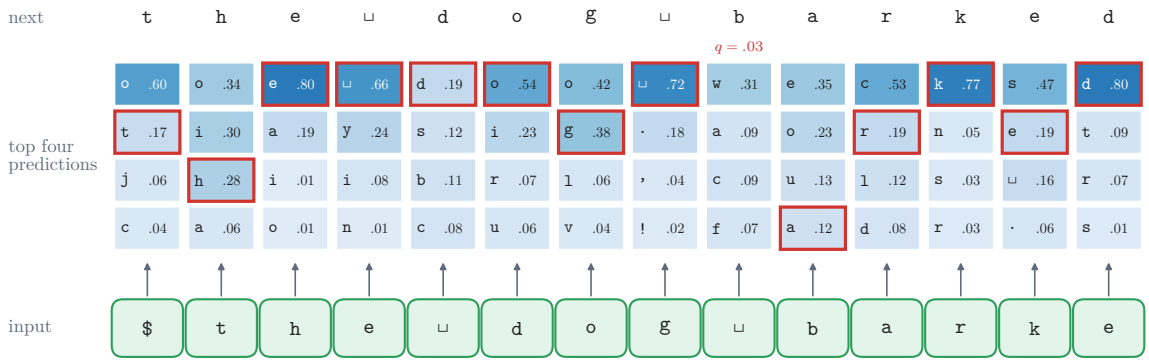


Figure 16. A trained recurrent network (the plain RNN of this section) reading "the dog barked". Each column is one step: the character read (bottom), the four most probable next characters with their probabilities (shaded by probability), and the character that actually comes next, outlined. After Karpathy's "hello" example, with a real model.

Training: backpropagation through time

The training loss is the log loss of every next character, $L = \sum_t L_t$ with $L_t = -\log q(x_{t+1} \mid x_{\le t})$. During training the network reads the true text, not its own samples, and predicts every next character along the way (**teacher forcing**).

The gradient is computed by backpropagation on the unrolled network, which for recurrent networks is called **backpropagation through time** (BPTT; Werbos, 1990). Nothing beyond Lecture 1's autograd engine is needed, and Figure 17, the computation graph of three steps, shows why. The weight matrices W, V, and U are **leaf nodes**, inputs that no other node computes, like the parameters of Lecture 1's engine. Weight sharing appears as fan-out: each matrix is one node, with an edge into every step that uses it. Backpropagation treats such a node like any value used more than once: it adds up the gradients arriving along its outgoing edges. So

$$\frac{\partial L}{\partial W} = \sum_t \Big(\text{the gradient through the use of } W \text{ at step } t\Big),$$

the gradient W would receive if each step had its own copy of it, summed over the copies. The same holds for V, U, and the word table C.

The gradient with respect to a state collects everything that state affects. In Figure 17, $\mathbf{h}_3$ affects only L_3; $\mathbf{h}_2$ affects L_2 directly and L_3 through $\mathbf{h}_3$; $\mathbf{h}_1$ affects all three. So the backward pass runs from the last step to the first (the red arrows), adding at each step the gradient of the local loss to the gradient arriving from the next state:

$$\frac{\partial L}{\partial \mathbf{h}_t} = \frac{\partial L_t}{\partial \mathbf{h}_t} + \left(\frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t}\right)^{\top} \frac{\partial L}{\partial \mathbf{h}_{t+1}}.$$

The second term carries the training signal backwards in time; the subsection after next follows it over many steps.

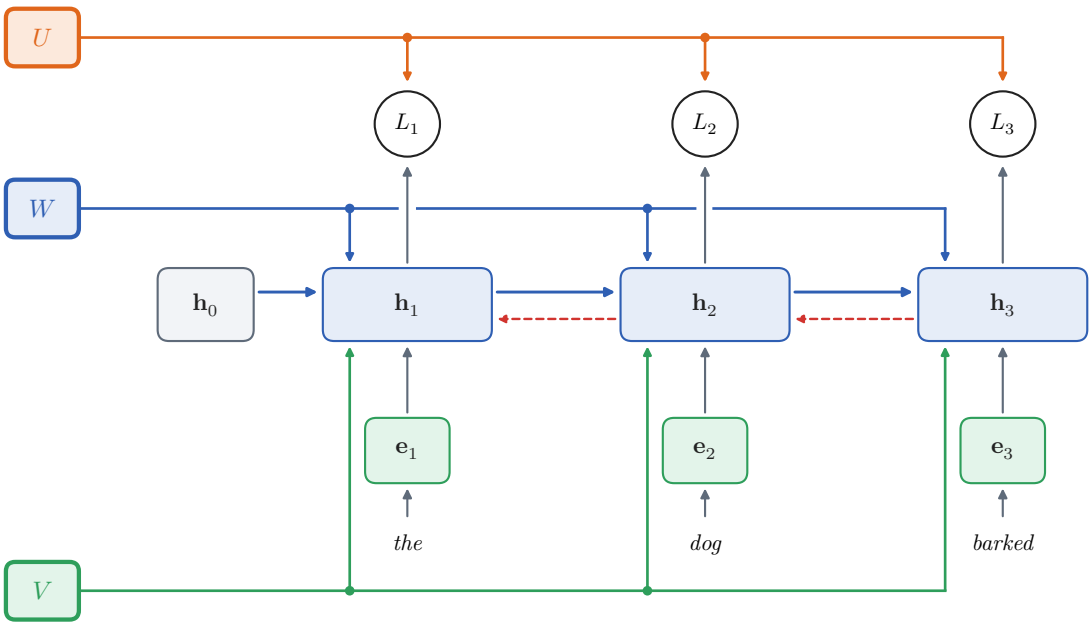


Figure 17. The computation graph of a recurrent network over three tokens, the graph that backpropagation through time differentiates. Step t computes $\mathbf{e}_t = C(x_t)$, the state $\mathbf{h}_t = \tanh(W\mathbf{h}_{t-1} + V\mathbf{e}_t)$, and the loss L_t of the prediction $\mathrm{softmax}(U\mathbf{h}_t)$ of the next token; the training loss is $L = L_1 + L_2 + L_3$. The weight matrices W, V, and U are leaf nodes, each with an edge into every step that uses it: this fan-out is the weight sharing, and the gradient of L with respect to each matrix is the sum of the gradients arriving along its edges. The red dashed arrows are the backward path through the states, along which the gradient of a later loss reaches an earlier state. (The word table C and the biases are leaves too, omitted.)

Two practical points follow from the unrolling. Memory grows with the length of the sequence, since every state is kept for the backward pass, so long texts are cut into chunks, and gradients do not flow from one chunk to the previous one (**truncated BPTT**). And the forward pass is sequential: $\mathbf{h}_t$ cannot be computed before $\mathbf{h}_{t-1}$, so the steps of a sequence cannot be spread across the thousands of parallel units of a GPU.

The models of these notes train on the same 6,000 stories as the character n-grams of Figure 4 and are evaluated on the same 500 held-out stories. Each step takes 64 random chunks of 128 characters, predicts every character of every chunk, and takes one Adam step, with the gradient rescaled if its norm exceeds 1 (clipping, explained below). 3,000 steps read about 25 million characters, five passes over the training text.

Generation

Generation is the loop with the model's output fed back in: start from the end-of-story symbol (as input it plays the role of BOS), sample the next character from $q(\cdot \mid x_{\le t})$, feed it in as the next input, and stop when the end-of-story symbol is sampled. The state carries everything the model will use, so each new character costs one step of the recurrence, however long the text so far. A sample from the plain RNN of this section, with 256 hidden units:

> jumped orange cookies, with toys on it. do you like toys and scared, but no home herself. the button't like it to the lake and pussed the sharp som'ther. he did not reivep up, catch in the hard growing and be potate as their mom hungry and gave his mom's bake. they wood zoom!" "thin …

Many words are spelled right and some phrases are TinyStories phrases, but many words are invented, and sentences hold together for only a few words.

Why the plain network falls short

The plain RNN, with 256 units and 84,000 parameters, reaches 1.47 bits per character on the held-out stories after 25 million characters of training, and has nearly stopped improving (Figure 21, left); the

best character n-gram model reaches 1.26. It could in principle use the whole prefix; in practice it learns to use only the recent past. The reason is in its gradients.

The product of Jacobians. How does the loss at step t depend on the state k steps earlier? Applying the backward recursion k times gives, by the chain rule,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \prod_{j=t-k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \prod_{j=t-k+1}^{t} \operatorname{diag}(1 - \mathbf{h}_j^{\odot 2})\, W,$$

a product of k Jacobians, each the weight matrix W followed by the slopes $1 - h^2$ of the tanh units ($\mathbf{h}_j^{\odot 2}$ squares each entry). For a single unit with a linear update, $h_t = w h_{t-1} + x_t$, the product is $\partial h_t / \partial h_{t-k} = w^k$: with $w = 0.5$ the influence of a token ten steps back is $0.5^{10} \approx 0.001$, with $w = 1.5$ it is $1.5^{10} \approx 58$. Unless w is very close to 1, a product of many factors either vanishes or explodes exponentially. In the matrix case the singular values of W play the role of w: if the largest is below 1, the gradient from k steps back is at most $\sigma_{\max}(W)^k$ times the gradient at the current step (§A.7). Sepp Hochreiter analyzed this in his 1991 diploma thesis, as did Bengio, Simard, and Frasconi (1994).

Saturated units. The trained network's W is not small: its largest singular value is 6.8, and half of them exceed 1.6. Its gradients vanish anyway, because of the other factor in each Jacobian, the slopes. The slope of unit i at step j is $1 - h_{j,i}^2$: 1 when the unit's value is 0, and nearly 0 when the value is near ± 1, where tanh is flat. A unit near ± 1 is **saturated**: its output barely changes when its input changes, and since its slope multiplies the gradient passing back through it, almost no gradient gets through.

Figure 18a shows how often this happens in the trained network. Read it as a picture of the state over time: each row is one of the 256 hidden units, each column one character of a held-out story, read left to right, and a pixel is white where that unit is saturated after that character ($|h| > 0.99$, slope below 0.02) and black elsewhere. Half of the pixels (51%) are white, and the average slope is 0.18. The white pixels change from column to column: which units are saturated changes with every character, so a gradient travelling back through many steps keeps meeting units that block it. (Karpathy draws the same picture in *Building makemore Part 3*; Part 6 of the notebook draws it for any text.)

What reaches the past. Figure 18b measures the result. Take a window of 160 characters of training text and the loss of the network's last prediction in it. The gradient of that loss with respect to the input vector $\mathbf{e}_t$ of an earlier character measures how much that character can change the prediction, and so how much training signal it receives. Each row of Figure 18b is one of 24 random windows, each column says how many characters back the input is, and the shade is the size of that gradient relative to the gradient for the latest character, on a logarithmic scale: white is 1 (as large as for the latest character), black is 10^{-12} or less. The median over the windows is 0.11 ten characters back, 0.01 twenty back, 2×10^{-6} fifty back, and 10^{-11} a hundred back: the signal shrinks about tenfold every ten characters. A name mentioned two sentences earlier is a few hundred characters back and receives essentially none.

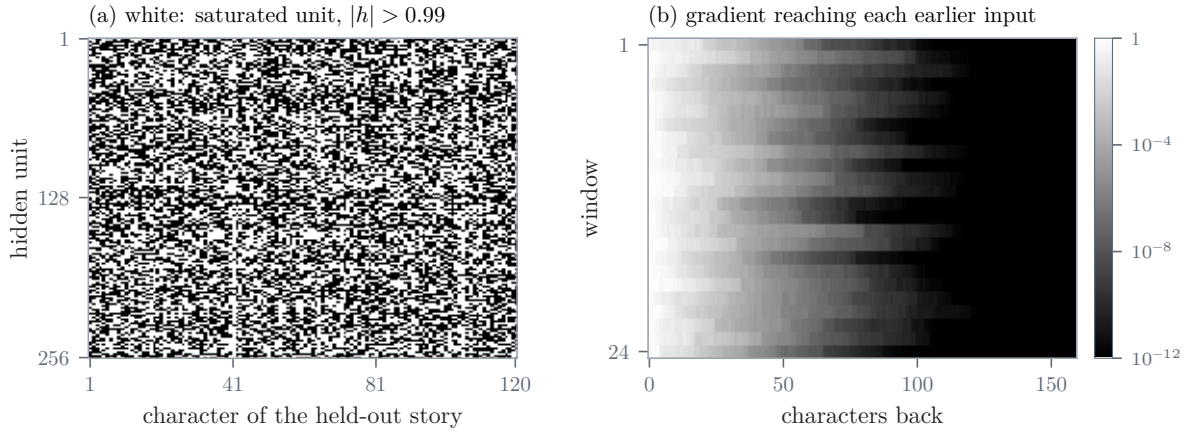


Figure 18. Vanishing gradients in the trained plain RNN. (a) Its 256 hidden units (rows) while it reads 120 characters of a held-out story (columns, left to right); a pixel is white where the unit is saturated after that character, $|h| > 0.99$, so that it passes back less than 2% of the gradient that reaches it, and black elsewhere. (b) For 24 windows of 160 characters (rows): the size of the gradient of the loss of the window's last prediction with respect to the input of each earlier character, divided by that for the latest character, against how many characters back the input is (columns). Logarithmic shading: white is 1, black 10^{-12} or less.

A memory test. Figure 18b shows that the signal from far back is tiny. Does that stop the network from learning to use a distant token? A simple task isolates the question. Each sequence is a **key**, a

symbol drawn from eight, then T noise symbols drawn at random from eight others, then a query symbol; at the query, and only there, the network must output the key. With $T = 8$, for example, the sequence 5, 12, 9, 14, 11, 8, 15, 13, 10, ? has answer 5. A network that stores the key in its state and keeps it for T steps is always right; one that does not can only guess, and is right one time in eight. For each gap T, a fresh plain RNN with 64 units is trained with the same budget (1,500 steps, each on 128 new random sequences, a few seconds to a minute) and then tested on 4,000 new sequences (Figure 19; Part 6 of the notebook runs the same test). With $T = 5$ it is always right, with $T = 10$ right 24% of the time, and from $T = 20$ on at chance. Training four times longer does not move the limit: after 6,000 steps it is right about 25% of the time at $T = 10$, 15, and 20. The network is large enough to represent the solution (copy the key into a few units and hold it there). What it lacks is a training signal: the gradient that would connect the loss at the query to the key is as small as Figure 18b shows, so the network never learns that storing the key would help. The other curve in Figure 19 is the LSTM of §8, trained the same way: it passes the test at every gap up to 80 and fails at 160. §8 explains both.

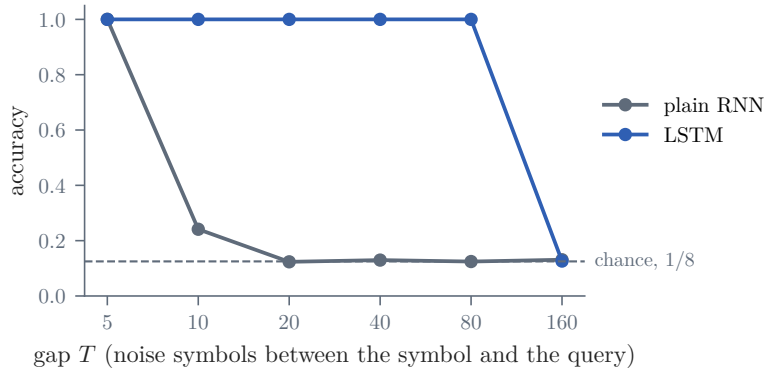


Figure 19. The memory test: remember the key, the first symbol of a sequence, across T noise symbols. Accuracy on 4,000 new sequences after training, for a plain RNN and an LSTM (§8) with 64 units each and the same training budget, against the gap T; chance is 1/8.

The two failures of a long product have different cures. An exploding gradient is a numerical problem, controlled by **gradient clipping**: if the norm of the gradient exceeds a threshold, rescale it to the threshold (Pascanu, Mikolov, and Bengio, 2013). A vanishing gradient is a modeling problem: nothing is wrong with the computation, but the network receives almost no training signal connecting an early token to a late prediction, so it never learns to use the dependency. Fixing it requires a different architecture.

8. Long short-term memory

Gated memory

The memory test shows what the plain network lacks. To pass it, a network must **write** the first symbol into its state, **keep** it unchanged while T noise symbols go by, and **read** it out at the query. The plain RNN can do none of these selectively: its update $\mathbf{h}_t = \tanh(W\mathbf{h}_{t-1} + V\mathbf{e}_t + \mathbf{d})$ rewrites the whole state at every step, so each noise symbol mixes with what is stored, and what is stored passes through W and the tanh and changes a little. The gradient travels the same path backwards, and Figure 18 showed that it mostly shrinks.

A memory that stays put. Hochreiter and Schmidhuber's remedy (1997) separates memory from computation. Next to the state $\mathbf{h}_t$, the network keeps a second vector, the **cell** $\mathbf{c}_t$, which is changed only by adding to it:

$$\mathbf{c}_t = \mathbf{c}_{t-1} + (\text{new content}).$$

Without new content, the stored value passes from step to step unchanged, with no weight matrix and no squashing on the way, and so does its gradient, since $\partial\mathbf{c}_t/\partial\mathbf{c}_{t-1}$ is the identity. This is the scalar example of §7 with $w = 1$ exactly, the one value at which the product neither vanishes nor explodes; Hochreiter and Schmidhuber called it the *constant error carousel*.

Switches for every flow. A cell that adds everything it reads would fill up with noise, and a cell that is always visible would disturb predictions with content that does not matter yet. So each flow into and out of the cell passes through a **gate**: a vector of numbers between 0 and 1, computed by a sigmoid from the previous state and the current input, that multiplies the flow entry by entry, a switch for each

unit that the network opens or closes depending on what it reads. There is one gate for each thing the memory test asks for:

$$\mathbf{i}_t = \sigma(W_i \mathbf{h}_{t-1} + V_i \mathbf{e}_t + \mathbf{b}_i) \qquad \text{input gate: how much new content to write}$$
$$\mathbf{f}_t = \sigma(W_f \mathbf{h}_{t-1} + V_f \mathbf{e}_t + \mathbf{b}_f) \qquad \text{forget gate: how much of the old cell to keep}$$
$$\mathbf{o}_t = \sigma(W_o \mathbf{h}_{t-1} + V_o \mathbf{e}_t + \mathbf{b}_o) \qquad \text{output gate: how much of the cell to read out}$$
$$\mathbf{g}_t = \tanh(W_g \mathbf{h}_{t-1} + V_g \mathbf{e}_t + \mathbf{b}_g) \qquad \text{candidate content}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

In the memory test the network can open the input gate at the first symbol and close it for the noise, hold the forget gate near 1 throughout, and open the output gate at the query. The 1997 network had only input and output gates, so its cells could only accumulate; Gers, Schmidhuber, and Cummins (2000) added the forget gate, so that the network can also erase what it no longer needs. Figure 20 draws one step.

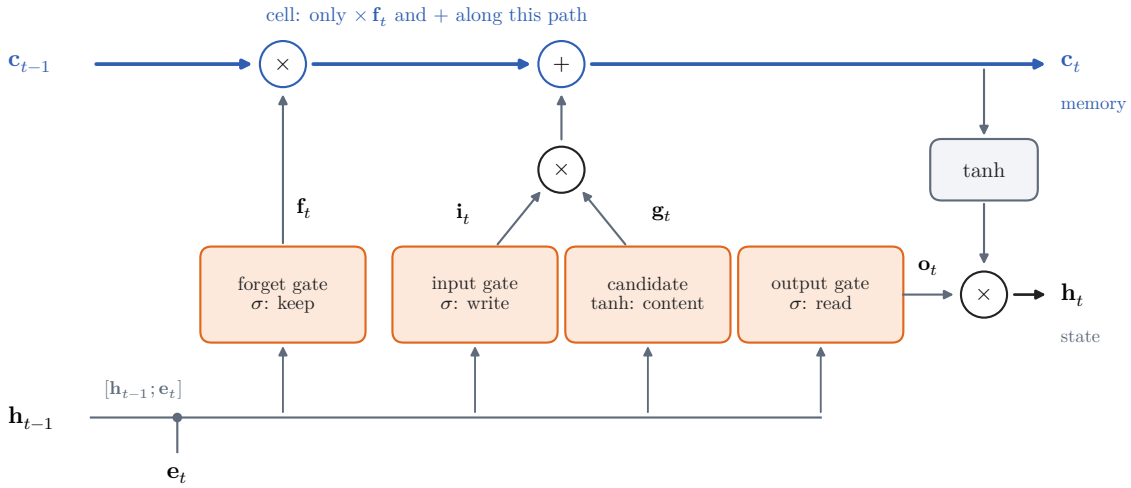


Figure 20. One step of an LSTM. The previous state $\mathbf{h}_{t-1}$ and the input vector $\mathbf{e}_t$ feed three gates and the candidate content. Along the cell line at the top, the old cell $\mathbf{c}_{t-1}$ is only scaled by the forget gate and added to, so information and gradients can pass through many steps; the output gate decides how much of the cell reaches the state $\mathbf{h}_t$, which the rest of the network sees.

The name. Psychology distinguishes long-term memory, what is learned and kept, from short-term memory, what is held in mind during a task. In a network, long-term memory is the weights, which change only during training, and short-term memory is the state, which changes with every token. A plain RNN's short-term memory fades within a few dozen steps (Figure 18b); the LSTM's can last long: a *long short-term memory*. The names of the parts say what each can do, not what it will be used for; which units store what is learned from the data, as the quotation cell below shows.

Why it trains. Along the direct path from $\mathbf{c}_{t-1}$ to $\mathbf{c}_t$ the Jacobian is $\mathrm{diag}(\mathbf{f}_t)$: no weight matrix and no saturating slope, only an elementwise factor that the network sets. Over k steps it is the product of the forget gates (§A.7). Where a forget gate stays near 1, the gradient from a late loss reaches an early input almost undiminished, so the network receives the signal that storing something would have helped, and can learn to store it.

Initialization matters. If the forget gates start near 1/2 (the sigmoid of 0, PyTorch's default), the product of twenty of them is 10^{-6}, and the LSTM fails the memory test at $T = 20$ like the plain network. Starting their bias at a positive value, so that a new network keeps most of its cell from step to step, is standard practice (Jozefowicz, Zaremba, and Sutskever, 2015); Figure 19 uses 3, which keeps 95% per step. The same arithmetic explains the failure at $T = 160$: $0.95^{160} \approx 3 \times 10^{-4}$, so at the start of training the gradient from the query barely reaches the first symbol. The LSTM does not abolish the product of many factors; it makes each factor a number the network can hold close to 1. The character LSTM of Figure 21 kept PyTorch's default and learned to hold its memory anyway, because in text much of what matters is near, which supplies a training signal from the start: on held-out text a third of its forget-gate values end up above 0.9, and 13% above 0.99. The memory test offers no such help, since nothing near the query predicts the answer.

The rest of the model is unchanged: the state $\mathbf{h}_t$ feeds the softmax, training is backpropagation through time (§7), and generation is the same loop. An LSTM layer has four times the parameters of a plain recurrent layer of the same width, $4h(h + m + 1)$, one set for each gate and one for the candidate.

The **gated recurrent unit** (GRU; Cho et al., 2014) is a simplified LSTM with similar results: it merges the cell into the state, ties writing to forgetting (what is written replaces a matching share of what is kept), and drops the output gate, so it has three sets of weights instead of four (§A.8).

The same idea, a path along which information and gradients are added rather than repeatedly multiplied, later made very deep feed-forward networks trainable. Highway networks (Srivastava, Greff, and Schmidhuber, 2015) gate an identity path across layers, as the LSTM does across time, and residual networks (He et al., 2015) drop the gate, so that each block computes $\mathbf{x} + F(\mathbf{x})$. Every layer of the Transformer has two such connections (Week 4).

The LSTM on TinyStories

Figure 21 compares the plain recurrent network with an LSTM of the same 256 units, trained identically. Both are a few dozen lines of PyTorch in Part 6 of the notebook. After the same 25 million characters (nine minutes on two CPU cores), the plain RNN has stalled at 1.47 bits per character on the held-out stories; the LSTM, with 307,000 parameters, is at 1.31 and still improving.

The gradients show why (Figure 21, right). For the LSTM, an input 50 characters back still receives 6×10^{-4} of the gradient of the latest input, and one 100 characters back 3×10^{-5}, against 2×10^{-6} and 10^{-11} for the plain RNN. A network cannot learn to use what it receives no gradient from.

Against counting. Is 1.31 better than counting? Size for size, by a wide margin. An n-gram model stores a count for every distinct string of up to n characters that occurs in training, so the fair comparison is at equal storage. The 6-gram model stores 399,000 counts, a little more than the LSTM's 307,000 parameters, and scores 1.43; the 7-gram, with 881,000 counts, scores 1.34. Counting needs about ten times the storage to match the LSTM: the 9-gram (2.8 million counts) scores 1.27 and the 10-gram (4.3 million) 1.26, the best any order achieves. There the two part ways. The n-gram model has reached its limit: one more character of context makes it worse, and only more data would help. The LSTM's limit is set by its size and training time, both of which can be raised on the same data. An LSTM with 512 units, trained twice as long on the same 6,000 stories (1.1 million parameters, about an hour on two CPU cores), matches the 10-gram after 1,250 steps and ends at 1.12 bits per character after 6,000: below every n-gram model, with about a quarter of the best one's storage (Figure 21, left; Part 6 of the notebook loads it).

At scale the gap is large. On the One Billion Word benchmark of news text, the Kneser–Ney 5-gram model scores a perplexity of 67.6 (Chelba et al., 2013) and a large LSTM 30.0 (Jozefowicz et al., 2016): as uncertain as a choice among 30 words, where the best count model is as uncertain as a choice among 68. In the same years LSTMs moved into products: Google's voice search in 2015, and Google Translate in 2016, whose neural system, eight LSTM layers for reading and eight for writing, made 60% fewer translation errors than the phrase-based system it replaced, as judged by human raters (Wu et al., 2016).

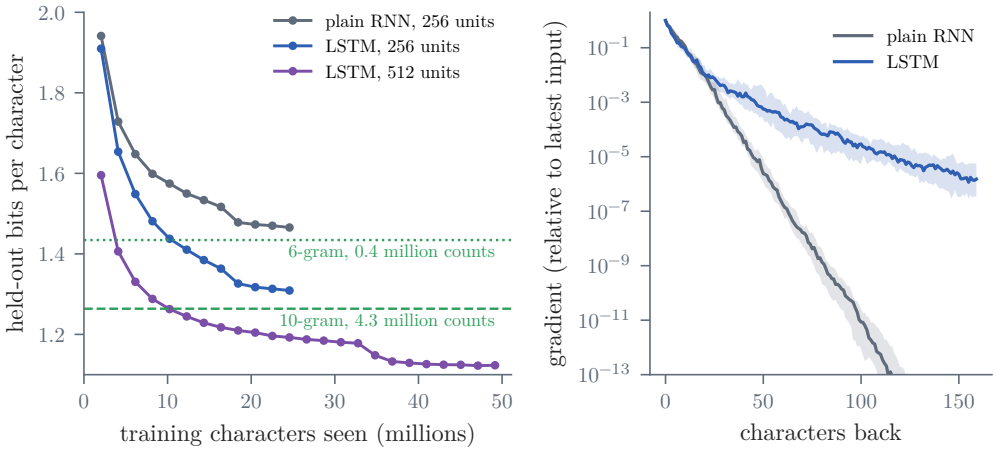


Figure 21. Recurrent character models of TinyStories against counting. Left: held-out bits per character during training, for a plain RNN and LSTMs of 256 and 512 units, against two character n-gram models of Figure 4: the best ($n = 10$) and one of about the 256-unit LSTM's size ($n = 6$). Right: the gradient of the last prediction's loss with respect to each earlier input, relative to the latest input (median over the 24 windows of Figure 18b, with the middle half shaded).

Watching it learn

Karpathy's essay printed samples from a model as it trained. Here is the same experiment for the 256-unit LSTM: one sample, always drawn with the same random numbers, after increasing amounts of training; the last row uses the same random numbers for the 512-unit LSTM.

After	**Held-out loss**	**Sample**
0 steps	5.08 bits per character	yuj?nvfaslgt"ykzir hfd
100 steps (0.8 million characters)	2.49 bits per character	yucind as gome, in the pupptle in the too ba ont cougerte a saw hwaws momer slemat. the thirs to twat a cark a lithe oul and omw her freppyigry. "winen, "is heme cras mean on wyst thit ware. hit the bill be thorun he suing lily soine pand a
300 steps (2.5 million characters)	1.83 bits per character	yuck afas of you rone with them, the can mia on the geat to saw aways mom. she was so take the bittle and book looked up at upefule. from your happine with her bocks and shoped still time,
1,000 steps (8.2 million characters)	1.48 bits per character	yucked as here, in the park. every happy, back to the barras and when she realized to take the old man was so ellore splashings. he cries your hopping with him. wasts all his wyite girl and had a decost once up whene out. from their apapero
3,000 steps (24.6 million characters)	1.31 bits per character	once upon a time, there was an incredible back to the barras away. when she slept and take them from hard and jumpsing. later, when she was so happy and happy me cars all the wyate?" timmy smiled and said, "thank you our guning so nearby?"
6,000 steps, 512 units (49 million characters)	1.12 bits per character	once upon a time, there was an intelligent bear. he was a wisd carefully trusted the flashith. he was a bear and a fish. at the party, they started to walk outside and laughed you today. max had talked to her daughter. it felt something was

The untrained model writes random characters and soon stops, since the end-of-story symbol is one of 34 equally likely choices. After 100 steps it has learned the letter frequencies, that words are separated by spaces, many short words (*in*, *the*, *a*, *saw*), and that quotation marks exist. After 300 steps most common words are spelled right and names appear. After 1,000 steps the phrases are TinyStories phrases (*in the park*, *when she realized*), and after 3,000 the sample opens with *once upon a time, there was* and contains dialogue (*timmy smiled and said*). In the 512-unit model's sample most words are real and most sentences grammatical, though they do not yet add up to a story.

A lower temperature (next subsection) trades variety for coherence. The most coherent of five samples from the 512-unit LSTM at temperature 0.7:

> once upon a time, there was a little girl named lily. she loved to play outside and look for other onions. one day, while they were playing, lily noticed something even better. she thanked the man and said, "thank you, tom. you are my friend. we can share the cookies together." sam thought for a moment and then said, "don't worry, we can use them again?" the bird said, "yes, please. thank you, mom …

It has a named character, a setting, and dialogue with matching quotation marks, but it loses track of who is who: *lily* thanks a man she calls *tom*, *sam* answers her, and then a bird.

Temperature

Temperature (§6) trades accuracy for variety. Samples from the 256-unit LSTM, starting from the end-of-story symbol:

Temperature	Sample
0.5	one day, a little girl was so happy and didn't know what to do. she was so happy to see a big box of billy's voice. "hi hould not see it both carefully me to play with you. you are sorry for you." mom said, "yes, i have to go …
1.0	greezing along to searching together on the rain. she saw a smelly set off and wanted to come back but lily's mom went back to play. she picked up her, but before loved to recording body was very three years old and piptent it …
1.5	greezing. he trides and drawbo. doal orry valuable heay!" ben caughing your house!" sam nodded, their noises vet, behind the wap. suddenly, but soonth, "let?" mommy reived the craps, of comings groms about ben, grash approinie …

At 0.5 the model keeps to its most probable continuations, and the text is fluent and repetitive (*so happy* twice in two sentences). At 1.0 it samples from the distribution it learned, the one the loss scores. At 1.5 it gives too much weight to unlikely characters, and words fall apart. The samples at 1.0 and 1.5 begin alike because they use the same random numbers: raising the temperature changes the choices only where the distribution is uncertain.

What the cells track

Do individual parts of a trained LSTM mean anything? Karpathy, Johnson, and Fei-Fei (2015) found, in LSTMs trained on novels and source code, cells that switch on inside quotations, cells that count the characters of a line, and cells that track the depth of nested brackets. The search is easy to repeat on TinyStories, which is full of dialogue.

What is measured. The cell $\mathbf{c}_t$ of the 256-unit LSTM is a vector of 256 numbers, recomputed after every character. Each coordinate is a **cell unit**; cell unit 28 is the coordinate with index 28 (counting from 0), and as the model reads it traces out one number per character. Run the model over 200 held-out stories (175,000 characters) and record all 256 units after each character. Mark each character 1 if it lies inside a quotation, from an opening quotation mark up to its closing mark, and 0 otherwise; 13% of the characters are inside. For each unit, compute the correlation over the 175,000 characters between its value and the mark: near 1 for a unit that is high exactly inside quotations and low elsewhere, near 0 for a unit unrelated to them.

What is found. One unit stands out: cell unit 28 has correlation 0.79, the next best 0.58, and every other unit at most 0.32. Figure 22a shows unit 28 on held-out text: it jumps up at an opening quotation mark, stays high through the quotation, and drops at the closing mark. Figure 22b shows its values over all 175,000 characters: around -2 outside quotations and around $+3$ inside, with little overlap. The network was never told about quotation marks. It learned to track them because the next character depends on them: inside a quotation *i* and *you* are likely and a closing mark must come eventually; after one, *said* and *asked* are likely.

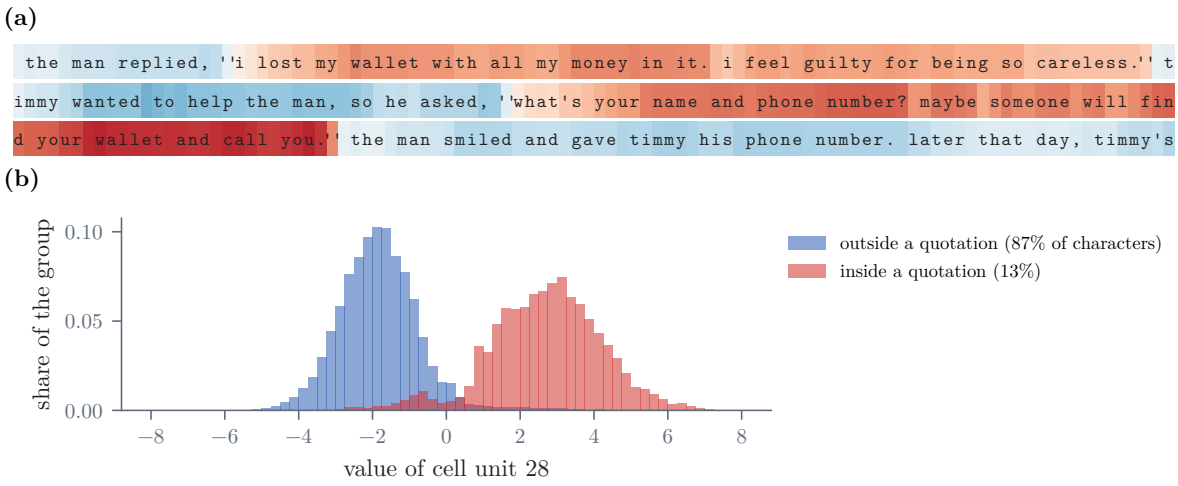


Figure 22. A cell unit of the TinyStories LSTM that tracks quotations. (a) Held-out text, each character shaded by the value of cell unit 28 after reading it (blue negative, red positive). (b) The distribution of the unit's value over 175,000 held-out characters, separately for characters inside and outside a quotation. Nobody told the network about quotation marks; this unit learned to keep track of being inside one, because the characters that come next depend on it.

Words instead of characters

The same LSTM works on words: replace the character vectors by the word table C and the 34-way softmax by an 8,681-way softmax, and train on the word stream. Sized like the fixed-window model (64-dimensional word vectors and 128 units: 1.8 million parameters against its 1.7 million), it trains on random chunks of 64 words for 4,000 steps, about 16 million words or four and a half passes over the training stories, in about half an hour on two CPU cores. Each test story is read from a zero state and scored on the same prediction events as the n-gram models. Its test perplexity is 23.1: better than the fixed-window network with the same budget of parameters, and better than the Kneser–Ney 4-gram model, which no single model of §6 beat; and it was still improving when training stopped. (Averaging the fixed-window network with the 4-gram model gave 21.4 in §6: neural and count models make different errors.) The model reads the whole story so far, and what it learns at one position applies at all of them, which the n-gram models could never do.

Model	Context	Parameters	Test perplexity
Kneser–Ney 4-gram	three words	counts	24.6
Fixed-window network	four words	1.7 million	26.9
LSTM	the whole story so far	1.8 million	23.1

Where recurrent networks were used

For most of their first two decades recurrent networks stayed in research, with successes in handwriting recognition (Graves et al., 2009) and speech recognition (Graves, Mohamed, and Hinton, 2013). From about 2014 to 2018 they were the standard model for anything sequential. Google's voice search moved to LSTM acoustic models in 2015 (Google Research, 2015); Google Translate replaced its phrase-based system with eight encoder and eight decoder LSTM layers in 2016, cutting translation errors by about 60% in human ratings (Wu et al., 2016); Facebook's 4.5 billion translations a day ran on LSTMs with attention by 2017 (Facebook Engineering, 2017); and phone keyboards predicted the next word with LSTMs (Hard et al., 2018). Transformers then replaced them: GPT and BERT (2018) are Transformers, and by 2020 Google Translate used a Transformer encoder with a recurrent decoder (Google Research, 2020).

What recurrence leaves unsolved

Two limitations remain. First, the state is a fixed-size vector, and everything the model will need about the prefix must be squeezed into it. This became visible in machine translation, where an **encoder–**

decoder pair of LSTMs read a source sentence into one vector and generated the translation from it (Sutskever, Vinyals, and Le, 2014): quality fell on long sentences. Bahdanau, Cho, and Bengio (2015) let the decoder look back at all the encoder's states, weighted by relevance, instead of relying on the last one. Second, the computation is sequential in time, which wastes parallel hardware and makes training on very large corpora slow. The mechanism that answers both, attention, does away with recurrence altogether; it is where the next lecture begins.

9. What carries forward

Much of this lecture survives, in another form, in every modern language model.

- **The object.** A distribution over token sequences, factorized by the chain rule, with an end-of-sequence token and its mass on finite strings. What a token is (§1) is decided before any modeling.
- **The objective.** Maximum likelihood, minimizing log loss on the training text, is how every model in the course is pretrained. Counting and gradient descent find the same maximum when the model is a table (§6); what changes from model to model is the family over which the likelihood is maximized.
- **The evaluation.** Cross-entropy, perplexity, and bits per byte, with the same rules: score the same tokens, split by document, choose on validation, report on test.
- **Smoothing as inductive bias.** A model without smoothing has memorized its training data. Every way of giving probability to unseen events encodes a belief about which are plausible: add-α believes in uniformity, Kneser–Ney in versatile words, a neural model in the geometry of its learned vectors and, through regularization, in small weights.
- **Embeddings.** Words, and later every token, are vectors learned so that things used alike are near each other. Every model in the rest of the course starts with such a table, and the same construction underlies semantic search and vector databases.
- **Sharing.** The decisive step was to stop treating contexts as unrelated. Parameters shared across words (§6), then across positions (§7), let evidence transfer from seen contexts to unseen ones, which a table cannot do.
- **Paths for gradients.** A network can only learn to use information its gradients can reach. The LSTM's additive cell was the first architecture built around that fact, and not the last.

10. Background reading

The basics of §§2–8 are developed in full elsewhere. The table lists where: the chapters of the Background Notes on Machine Learning and AI, standard textbooks, and code-along lectures.

Topic	Background Notes	Textbook
n-gram models, smoothing, and perplexity	NLP & LLMs 2	Jurafsky and Martin, *N-gram Language Models*; Chen and Goodman (1998), the technical report
Entropy, cross-entropy, and coding	Foundations 5	Cover and Thomas, Chapters 2 and 5
Tokens and tokenizers	NLP & LLMs 1	Karpathy, *Let's build the GPT Tokenizer* (video)
Word vectors	NLP & LLMs 3	Jurafsky and Martin, *Embeddings*
Vector-space retrieval and nearest-neighbor search		Manning, Raghavan, and Schütze, *Introduction to Information Retrieval*, Chapters 6 and 18
Neural language models, built in code		Karpathy, *Neural Networks: Zero to Hero*, the two makemore lectures (videos)
Recurrent networks and LSTMs	DL 8	Goodfellow, Bengio, and Courville, Chapter 10
Hidden Markov models (§A.6)	AI 11	Rabiner (1989); Jurafsky and Martin, appendix on HMMs

Topic	Background Notes	Textbook
The EM algorithm (§A.6)	ML 14	Bishop, *Pattern Recognition and Machine Learning*, Chapter 9

For reading in the order of this lecture: Shannon (1951) is short and still the best introduction to the entropy of English; the n-gram chapter of Jurafsky and Martin covers §§2–4 with more examples; Mikolov et al. (2013b) and Levy and Goldberg (2014) are short and together tell the story of §5; Karpathy's two makemore lectures build §6 in code, and Bengio et al. (2003) is a readable paper whose first two sections state the curse of dimensionality for language modeling and the idea of the solution; and the essays of Karpathy and Olah (in the references) are the best short introductions to recurrent networks and LSTMs (§§7–8).

References and further reading

Stochastic sources

- **Andrey A. Markov (1913), An Example of Statistical Investigation of the Text *Eugene Onegin* Concerning the Connection of Samples in Chains.** English translation, *Science in Context* 19(4), 2006.
- **Brian Hayes (2013), First Links in the Markov Chain.** Article. The history, and Markov's counts.
- **Claude E. Shannon (1948), A Mathematical Theory of Communication.** Paper. The approximations to English are in Section 3.
- **Claude E. Shannon (1951), Prediction and Entropy of Printed English.** Paper.

n-gram models, smoothing, and coding

- **I. J. Good (1953), The Population Frequencies of Species and the Estimation of Population Parameters.** Paper. Good–Turing estimation, credited to Turing.
- **Frederick Jelinek and Robert L. Mercer (1980), Interpolated Estimation of Markov Source Parameters from Sparse Data.** In *Pattern Recognition in Practice*, North-Holland.
- **Slava M. Katz (1987), Estimation of Probabilities from Sparse Data for the Language Model Component of a Speech Recognizer.** Paper. Backoff.
- **Ian H. Witten, Radford M. Neal, and John G. Cleary (1987), Arithmetic Coding for Data Compression.** Paper.
- **Reinhard Kneser and Hermann Ney (1995), Improved Backing-Off for M-Gram Language Modeling.** Paper.
- **Stanley F. Chen and Joshua Goodman (1999), An Empirical Study of Smoothing Techniques for Language Modeling.** Paper. The standard comparison; modified Kneser–Ney.
- **Yee Whye Teh (2006), A Hierarchical Bayesian Language Model Based on Pitman–Yor Processes.** Paper. Kneser–Ney as approximate Bayesian inference.
- **Thorsten Brants et al. (2007), Large Language Models in Machine Translation.** Paper. Stupid backoff on two trillion tokens.

Word embeddings

- **Charles E. Osgood, George J. Suci, and Percy H. Tannenbaum (1957), The Measurement of Meaning.** University of Illinois Press. Word meanings as points in a space of rated scales.
- **Gerard Salton, Anita Wong, and Chung-Shu Yang (1975), A Vector Space Model for Automatic Indexing.** Paper.
- **Geoffrey E. Hinton (1986), Learning Distributed Representations of Concepts.** *Proceedings of the Eighth Annual Conference of the Cognitive Science Society.* The family-tree network.

- **Kenneth W. Church and Patrick Hanks (1990), Word Association Norms, Mutual Information, and Lexicography.** Paper.
- **Scott Deerwester et al. (1990), Indexing by Latent Semantic Analysis.** Paper.
- **Thomas K. Landauer and Susan T. Dumais (1997), A Solution to Plato's Problem: The Latent Semantic Analysis Theory of Acquisition, Induction, and Representation of Knowledge.** Paper.
- **Dekang Lin (1998), Automatic Retrieval and Clustering of Similar Words.** Paper. Similar words from shared contexts; the *tezgüino* example, after Nida (1975).
- **John A. Bullinaria and Joseph P. Levy (2007), Extracting Semantic Representations from Word Co-occurrence Statistics: A Computational Study.** Paper.
- **Ronan Collobert and Jason Weston (2008), A Unified Architecture for Natural Language Processing: Deep Neural Networks with Multitask Learning.** Paper.
- **Michael Gutmann and Aapo Hyvärinen (2010), Noise-Contrastive Estimation: A New Estimation Principle for Unnormalized Statistical Models.** Paper.
- **Ronan Collobert et al. (2011), Natural Language Processing (Almost) from Scratch.** Paper.
- **Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean (2013a), Efficient Estimation of Word Representations in Vector Space.** Paper. CBOW and skip-gram.
- **Tomas Mikolov et al. (2013b), Distributed Representations of Words and Phrases and Their Compositionality.** Paper. Skip-gram with negative sampling.
- **Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig (2013), Linguistic Regularities in Continuous Space Word Representations.** Paper. The analogy test.
- **Omer Levy and Yoav Goldberg (2014), Neural Word Embedding as Implicit Matrix Factorization.** Paper.
- **Jeffrey Pennington, Richard Socher, and Christopher D. Manning (2014), GloVe: Global Vectors for Word Representation.** Paper.
- **Omer Levy, Yoav Goldberg, and Ido Dagan (2015), Improving Distributional Similarity with Lessons Learned from Word Embeddings.** Paper.
- **Tolga Bolukbasi et al. (2016), Man Is to Computer Programmer as Woman Is to Homemaker? Debiasing Word Embeddings.** Paper.
- **Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov (2017), Enriching Word Vectors with Subword Information.** Paper. fastText.
- **Matthew E. Peters et al. (2018), Deep Contextualized Word Representations.** Paper. ELMo: one vector per occurrence.

Embeddings in search, retrieval, and recommendation

- **Po-Sen Huang et al. (2013), Learning Deep Structured Semantic Models for Web Search Using Clickthrough Data.** Paper.
- **Yury A. Malkov and Dmitry A. Yashunin (2016), Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.** Paper. HNSW graphs.
- **Jeff Johnson, Matthijs Douze, and Hervé Jégou (2017), Billion-Scale Similarity Search with GPUs.** Paper. The FAISS library.
- **Mihajlo Grbovic and Haibin Cheng (2018), Real-Time Personalization Using Embeddings for Search Ranking at Airbnb.** Paper.
- **Nils Reimers and Iryna Gurevych (2019), Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks.** Paper.
- **Vladimir Karpukhin et al. (2020), Dense Passage Retrieval for Open-Domain Question Answering.** Paper.

Neural language models

- **Andrej Karpathy (2022–2024), Neural Networks: Zero to Hero.** Playlist. Code-along lectures from backpropagation to a GPT; the course follows their spirit. *The spelled-out intro to language modeling: building makemore* (Video) builds the counting and neural bigram models of §6, *Building makemore Part 2: MLP* (Video) the fixed-window model, for the letters of names, and *Building makemore Part 3: Activations & Gradients, BatchNorm* (Video) shows saturated tanh units and what they do to gradients (§7).
- **Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin (2003), A Neural Probabilistic Language Model.** Paper.
- **Jeffrey L. Elman (1990), Finding Structure in Time.** Paper.
- **Paul J. Werbos (1990), Backpropagation Through Time: What It Does and How to Do It.** Paper.
- **Yoshua Bengio, Patrice Simard, and Paolo Frasconi (1994), Learning Long-Term Dependencies with Gradient Descent Is Difficult.** Paper.
- **Sepp Hochreiter and Jürgen Schmidhuber (1997), Long Short-Term Memory.** Paper.
- **Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins (2000), Learning to Forget: Continual Prediction with LSTM.** Paper.
- **Tomas Mikolov et al. (2010), Recurrent Neural Network Based Language Model.** Paper.
- **Diederik P. Kingma and Jimmy Ba (2015), Adam: A Method for Stochastic Optimization.** Paper. The optimizer used for every neural model in these notes.
- **Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio (2013), On the Difficulty of Training Recurrent Neural Networks.** Paper. Gradient clipping.
- **Kyunghyun Cho et al. (2014), Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation.** Paper. The GRU.
- **Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio (2014), Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling.** Paper. LSTM against GRU (§A.8).
- **Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever (2015), An Empirical Exploration of Recurrent Network Architectures.** Paper. Over ten thousand variants of gated networks; the forget-gate bias.
- **Ciprian Chelba et al. (2013), One Billion Word Benchmark for Measuring Progress in Statistical Language Modeling.** Paper. The benchmark and its Kneser–Ney baseline, perplexity 67.6.
- **Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu (2016), Exploring the Limits of Language Modeling.** Paper. Large LSTM language models: perplexity 30.0 on the same benchmark.
- **Yonghui Wu et al. (2016), Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation.** Paper. LSTMs in production translation.
- **Ilya Sutskever, Oriol Vinyals, and Quoc V. Le (2014), Sequence to Sequence Learning with Neural Networks.** Paper.
- **Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio (2015), Neural Machine Translation by Jointly Learning to Align and Translate.** Paper.
- **Andrej Karpathy (2015), The Unreasonable Effectiveness of Recurrent Neural Networks.** Essay. Character-level samples, and a visualization of what individual units track.
- **Andrej Karpathy, Justin Johnson, and Li Fei-Fei (2015), Visualizing and Understanding Recurrent Networks.** Paper. Interpretable LSTM cells: quotations, line length, bracket depth.
- **Christopher Olah (2015), Understanding LSTM Networks.** Essay. The standard pictures of the gates.

Hidden Markov models and the noisy channel (Appendix A.6)

- **Andrew J. Viterbi (1967), Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm.** Paper.

- **Leonard E. Baum, Ted Petrie, George Soules, and Norman Weiss (1970), A Maximization Technique Occurring in the Statistical Analysis of Probabilistic Functions of Markov Chains.** Paper.
- **Frederick Jelinek (1976), Continuous Speech Recognition by Statistical Methods.** Paper.
- **Frederick Jelinek (2004), Some of My Best Friends Are Linguists.** Invited talk, LREC 2004. Slides. The source of the "fire a linguist" remark, in his words.
- **Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin (1977), Maximum Likelihood from Incomplete Data via the EM Algorithm.** Paper.
- **Lawrence R. Rabiner (1989), A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition.** Paper.
- **Peter F. Brown et al. (1990), A Statistical Approach to Machine Translation.** Paper.
- **Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra, and Robert L. Mercer (1993), The Mathematics of Statistical Machine Translation: Parameter Estimation.** Paper.

Data and code

- **Alec Radford et al. (2019), Language Models are Unsupervised Multitask Learners.** Report. GPT-2, whose tokenizer of 50,257 tokens appears in Figure 1.
- **Ronen Eldan and Yuanzhi Li (2023), TinyStories: How Small Can Language Models Be and Still Speak Coherent English?** Paper. The corpus used for the measurements in these notes and in the course code.
- **Week 2 notebook.** Notebook, in the course codebase tinylm. Reproduces the measurements of these notes: n-gram models and the evaluation harness (`tinylm/ngram.py`), count vectors and word2vec (`tinylm/embeddings.py`), and the neural models (`tinylm/neural.py`), with trained checkpoints.

Appendix: deeper reading

This appendix collects proofs, derivations, and details that the main text states or sketches.

A.1 Markov's experiment on *Eugene Onegin*

The question. Pavel Nekrasov, a Moscow mathematician, had argued in 1902 that the law of large numbers requires independent trials, so that the stable averages of social statistics showed that the acts behind them were independent, which he took as evidence of free will. Markov disagreed. In 1906 he proved that the law of large numbers also holds for a chain, a sequence in which each trial depends on the one before: averages over long stretches still settle down to fixed values (Hayes, 2013). In 1913 he applied the theory to text, a source that is random but plainly not independent, and checked whether a chain fitted to it could predict its statistics.

The count. Markov wrote out the first 20,000 letters of the poem, without spaces or punctuation, and marked each as a vowel or a consonant: 8,638 vowels and 11,362 consonants, so a fraction $p = 0.432$ of the letters are vowels. Then he counted pairs of consecutive letters. If letters were independent, about $20{,}000 \times 0.432^2 \approx 3{,}730$ pairs would be two vowels; there were 1,104. Pairs of consonants were similarly rare: 3,827, against about 6,450. The letters alternate far more than chance would make them. From the counts, a vowel follows a vowel with probability $a = 1{,}104/8{,}638 \approx 0.128$ and follows a consonant with probability $b \approx 0.663$. These two numbers define a two-state chain (the transition matrix below has rows "from" and columns "to", vowel first):

$$P = \begin{pmatrix} 0.128 & 0.872 \\ 0.663 & 0.337 \end{pmatrix}.$$

The prediction. Markov then cut the 20,000 letters into 200 blocks of 100 consecutive letters (he wrote each block out as a 10×10 grid), counted the vowels x_i in each block, and computed their mean, about 43.2, and their variance, $\frac{1}{200}\sum_i (x_i - \bar{x})^2 \approx 5.1$. Under independence the count in a block would be binomial, with variance $100\,p(1-p) \approx 24.5$. The ratio of the two, which statisticians of the time

called the coefficient of dispersion, is 0.208: the blocks of the poem are far more alike than batches of coin tosses.

The chain predicts this ratio from a and b alone. Let X_t be 1 if letter t is a vowel and 0 otherwise, with the chain started in its stationary distribution, so that every X_t has mean $p = b/(1-a+b)$ and variance $p(1-p)$. Because X_t takes only the values 0 and 1, the expected next letter is a linear function of the current one, $\mathbb{E}[X_{t+1} \mid X_t] = b + \delta X_t$ with $\delta = a - b$. Stationarity gives $p = b + \delta p$, so $\mathbb{E}[X_{t+1} - p \mid X_t] = \delta\,(X_t - p)$: each step keeps a fraction δ of the current deviation from the mean. Applying this k times,

$$\mathbb{E}[X_{t+k} - p \mid X_t] = \delta^k\,(X_t - p), \qquad \text{so} \qquad \operatorname{Cov}(X_t, X_{t+k}) = \delta^k\,p(1-p).$$

Letters k apart have correlation δ^k; here $\delta = 0.128 - 0.663 = -0.535$, so neighbors are anticorrelated, letters two apart slightly correlated, and the dependence fades geometrically. In n letters there are $n-k$ pairs at distance k, so the number of vowels $S_n = X_1 + \cdots + X_n$ has variance

$$\operatorname{Var}(S_n) = p(1-p)\Big[n + 2\sum_{k=1}^{n-1}(n-k)\,\delta^k\Big] \;\approx\; n\,p(1-p)\,\frac{1+\delta}{1-\delta} \qquad (n \text{ large}).$$

The predicted ratio is $(1+\delta)/(1-\delta) = 0.465/1.535 \approx 0.30$ (0.307 with the exact sum for $n = 100$), a variance of about 7.5 against the binomial 24.5; 200,000 blocks simulated from the chain give the same 7.5. Each letter's deviation echoes down the chain with weights $\delta, \delta^2, \ldots$; with δ negative the echoes alternate in sign and largely cancel it. A chain with $\delta > 0$, whose states persist, would make the blocks vary more than coin tosses instead.

The second chain. The prediction gets the direction and most of the size, but not all of it: 0.30 against 0.208. The gap is not sampling noise, since a variance estimated from 200 blocks is uncertain by about 10%, roughly ± 0.02 on the ratio. Markov took it seriously and fitted a chain in which a letter depends on the previous two, its probabilities also counted from the poem. It predicts 0.195, within the noise of the observed value. He repeated the analysis on 100,000 letters of Sergei Aksakov's *The Childhood Years of Bagrov's Grandson*.

What it showed. For Markov the point was the theorem: a dependent sequence obeys the law of large numbers (here its block averages are even more stable than those of independent letters), so stable averages are no evidence of independence. For this lecture the point is the method. A model was fitted to local statistics, the counts of neighboring pairs, and checked against a property of the whole text it was not fitted to; a model with more context did better. That is the program of §§2–3: count, then evaluate on what the counts did not see. Shannon's 1948 paper models text as a "discrete Markoff process", but cites the probability literature (Fréchet's 1938 book), not the poem.

A.2 The Good–Turing estimates

Let N be the number of observed events and N_r the number of event types seen exactly r times. The **Good–Turing estimate** (Good, 1953, crediting Turing) of the probability that the next event is of a type never seen before is

$$\hat{p}_0 = \frac{N_1}{N},$$

the fraction of the sample made up of types seen once. The same reasoning replaces each count r by the adjusted count

$$r^* = (r+1)\frac{N_{r+1}}{N_r},$$

which is less than r for small r; the difference is the mass that pays for the unseen.

A check on TinyStories. Among the 3,559,907 bigram tokens of the training stories, $N_1 = 125{,}375$ bigram types occur exactly once, so Good–Turing predicts that 125,375/3,559,907 = 3.5% of new bigrams will be unseen; the test stories have 3.8%. For trigrams it predicts 17.9% against 18.3%. The adjusted counts also hold up: bigrams seen once in training occur on average 0.52 times in a test sample of the same size, against $r^* = 0.57$; bigrams seen twice occur 1.34 times, against $r^* = 1.44$. A count of 1 is an overestimate, by about half.

Derivation. Model the corpus as N independent draws from a distribution over types, type i having probability θ_i. The probability that the next draw is of a type not yet seen is $\sum_i \theta_i(1-\theta_i)^N$, whose expectation we want to estimate. The expected number of types seen exactly once is

$$\mathbb{E}[N_1] = \sum_i N\theta_i(1-\theta_i)^{N-1}.$$

Comparing the two expressions, $\mathbb{E}[N_1]/N = \sum_i \theta_i(1-\theta_i)^{N-1}$, which differs from the unseen mass only in replacing N by $N-1$. So N_1/N estimates the probability that draw $N+1$ is new, nearly without bias. The same computation for a type seen r times: the expected total probability of the types seen exactly r times is $\sum_i \theta_i \binom{N}{r} \theta_i^r (1-\theta_i)^{N-r}$, and

$$\mathbb{E}[N_{r+1}] = \sum_i \binom{N}{r+1} \theta_i^{r+1}(1-\theta_i)^{N-r-1},$$

so, again up to replacing N by $N \pm 1$ in the binomial factors, the total probability of the types seen r times is about $\frac{r+1}{N}\mathbb{E}[N_{r+1}]$. Dividing by the number N_r of such types gives each one probability $(r+1)N_{r+1}/(NN_r) = r^*/N$. For large r, N_{r+1} is small and noisy, so in practice the raw counts are used there, or the sequence N_r is smoothed first (Gale and Sampson's "simple Good–Turing"). McAllester and Schapire (*On the convergence rate of Good–Turing estimators*, COLT 2000) give high-probability bounds on the error of the missing-mass estimate.

A.3 Interpolation, backoff, and Kneser–Ney

Interpolation mixes the maximum-likelihood estimates of all orders (Jelinek and Mercer, 1980):

$$q_{\text{int}}(w \mid u, v) = \lambda_3\, q_{\text{MLE}}(w \mid u, v) + \lambda_2\, q_{\text{MLE}}(w \mid v) + \lambda_1\, q_{\text{MLE}}(w), \qquad \lambda_1 + \lambda_2 + \lambda_3 = 1,$$

with the weights chosen on held-out data, often separately for histories seen more or less often. **Backoff** (Katz, 1987) uses the higher-order estimate, discounted by Good–Turing, when the n-gram was seen, and otherwise falls back to the lower-order model, scaled so that the distribution still sums to one. At web scale even a crude version works: the "stupid backoff" of Brants et al. (2007), which does not normalize at all, was trained on two trillion tokens of web text for Google's machine translation system, in a paper titled *Large Language Models in Machine Translation*.

Kneser–Ney smoothing combines two ideas. The first is **absolute discounting**: subtract a fixed D (typically 0.75) from every positive count, and give the freed mass to the lower-order model. §A.2 explains the choice: adjusted counts are smaller than raw counts by a roughly constant amount for all but the smallest counts. The second idea concerns the lower-order distribution. The lower-order model is only consulted for histories where the higher-order evidence is thin, so it should estimate how likely w is to appear *in a new context*, and the right statistic is the number of distinct contexts w completes, the **continuation count** $N_{1+}(\bullet\, w) = |\{v : c(v, w) > 0\}|$. Interpolated Kneser–Ney for a bigram model is

$$q_{\text{KN}}(w \mid v) = \frac{\max(c(v,w) - D, 0)}{c(v)} + \frac{D\, N_{1+}(v\,\bullet)}{c(v)} \cdot \frac{N_{1+}(\bullet\, w)}{\sum_{w'} N_{1+}(\bullet\, w')},$$

where $N_{1+}(v\,\bullet)$ is the number of distinct words seen after v, so that the second term carries exactly the mass the discount removed. Higher orders apply the same formula recursively, with continuation counts at every order below the top. Chen and Goodman (1999) compared the smoothing methods systematically and found a modified Kneser–Ney, with three discounts, consistently best; Teh (2006) later showed that Kneser–Ney approximates inference in a hierarchical Pitman–Yor process, a Bayesian model of power-law distributions, which explains why it works.

A.4 When the conditionals define a distribution

Normalized conditionals do not automatically give a distribution over finite strings, because the sampling process might never emit EOS. Each finite string $x_{1:T}\text{EOS}$ receives the probability of §1, and these probabilities sum to the probability that sampling eventually stops. If that is less than one, the remaining mass sits on infinite sequences, and the model is called **inconsistent**, or **deficient**, on finite strings.

A simple sufficient condition rules this out. If $q(\text{EOS} \mid x_{<t}) \geq \varepsilon > 0$ for every prefix, then the probability of surviving m more steps is at most $(1-\varepsilon)^m \to 0$, so sampling stops with probability one. A softmax over a vocabulary that contains EOS, with bounded logits, always satisfies it. Positivity alone is not enough: if the probability of stopping at step t is $1/(t+1)^2$, the probability of never stopping is $\prod_{t \geq 1}(1 - 1/(t+1)^2) = 1/2$.

A.5 Skip-gram with negative sampling factorizes shifted PMI

§5 derives the optimum $\mathbf{v}_w \cdot \mathbf{u}_c = \log \frac{n(w,c)}{k\, n(w)\, p_n(c)}$ for each pair, under the assumption that every score can be set independently. Three remarks complete it.

The smoothed noise distribution. word2vec draws negatives from $p_n(c) = n(c)^{3/4} / \sum_{c'} n(c')^{3/4}$. Substituting it gives $\log \frac{p(w,c)}{p(w)\, p_n(c)} - \log k$: the PMI with the context's probability replaced by its smoothed version. The power 3/4 raises the probability of rare contexts, which lowers their PMI; this damps the large PMI values that rare pairs get by chance, a known weakness of plain PMI (Levy, Goldberg, and Dagan, 2015).

Pairs never observed. For $n(w, c) = 0$ only the negative term remains, and it is maximized at $s \to -\infty$, the analogue of PMI $= -\infty$. A low-dimensional factorization cannot reach that value; it gives the unseen pair a finite score determined by the vectors of similar words, which is one reason dense vectors generalize where the count matrix does not.

The expectation. The objective with sampled negatives is a random quantity; the derivation uses its expected value over the draws. Stochastic gradient descent follows the gradient of that expectation on average, so the optimum is the same.

A.6 Hidden Markov models and the noisy channel

Before neural networks, the main alternative to a Markov chain on the words themselves was a Markov chain on something the text does not show. This section develops that model and the use of language models inside larger systems.

Hidden Markov models

An n-gram model's state is the last few tokens, which the text shows. Many regularities of language are better described by a state the text does not show: whether the next word is likely to be a noun or a verb, which phoneme is being spoken, which topic a paragraph is about. A **hidden Markov model** (HMM) has K hidden states z_t that form a Markov chain, and each state emits an observation:

$$p(x_{1:T}, z_{1:T}) = \pi(z_1)\, B(z_1, x_1) \prod_{t=2}^{T} A(z_{t-1}, z_t)\, B(z_t, x_t),$$

with initial distribution π, transition matrix A, and emission distributions $B(z, \cdot)$. The probability of the observations alone sums over the K^T hidden sequences:

$$p(x_{1:T}) = \sum_{z_{1:T}} p(x_{1:T}, z_{1:T}).$$

Hidden states make an HMM more expressive than a Markov chain on the observations. The observed sequence of an HMM need not be Markov of any order, because the hidden state can carry information from arbitrarily far back.

The forward algorithm

The sum has K^T terms, but it factorizes. Let $\alpha_t(j) = p(x_{1:t}, z_t = j)$, the probability of the first t observations and of being in state j at time t. Then $\alpha_1(j) = \pi(j) B(j, x_1)$, and summing over the previous state,

$$\alpha_t(j) = \Big[\sum_{i=1}^{K} \alpha_{t-1}(i)\, A(i, j)\Big] B(j, x_t), \qquad p(x_{1:T}) = \sum_j \alpha_T(j).$$

Each step costs $O(K^2)$, so the whole computation takes $O(TK^2)$ time instead of $O(TK^T)$. This is dynamic programming: the probability of a long prefix is assembled from the probabilities of shorter ones, each computed once.

Take a two-state model of parts of speech, with states N (noun) and V (verb), and the two-word text *fish sleep*, in which each word could be either:

$$\pi = (0.8,\ 0.2), \qquad A = \begin{pmatrix} 0.2 & 0.8 \\ 0.7 & 0.3 \end{pmatrix}, \qquad B(\mathrm{N}, \cdot):\ \textit{fish}\ 0.7,\ \textit{sleep}\ 0.3; \qquad B(\mathrm{V}, \cdot):\ \textit{fish}\ 0.4,\ \textit{sleep}\ 0.6,$$

with rows and columns of A in the order N, V. The forward variables are $\alpha_1 = (0.8 \cdot 0.7,\ 0.2 \cdot 0.4) = (0.56,\ 0.08)$ and

$$\alpha_2(\mathrm{N}) = (0.56 \cdot 0.2 + 0.08 \cdot 0.7) \cdot 0.3 = 0.0504,$$
$$\alpha_2(\mathrm{V}) = (0.56 \cdot 0.8 + 0.08 \cdot 0.3) \cdot 0.6 = 0.2832,$$

so $p(\textit{fish sleep}) = 0.3336$ (Figure 23).

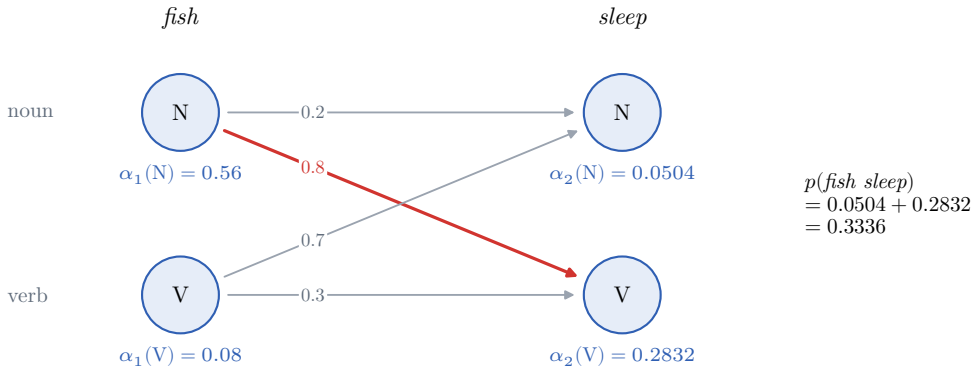


Figure 23. The forward algorithm on "fish sleep". Each node holds α_t, the probability of the words so far and the current state; edges carry transition probabilities. The red edge is the most probable path, N then V.

Decoding: the Viterbi algorithm

Which hidden sequence is most probable? Replace the sum in the forward recursion by a maximum, and remember which previous state achieved it: $\delta_t(j) = \max_i \delta_{t-1}(i) A(i,j)\, B(j, x_t)$. This is the **Viterbi algorithm** (Viterbi, 1967), again $O(TK^2)$. In the example $\delta_2(\mathrm{V}) = 0.56 \cdot 0.8 \cdot 0.6 = 0.2688$ and $\delta_2(\mathrm{N}) = 0.0336$, so the best tagging is *fish*/N *sleep*/V, with posterior probability $0.2688/0.3336 \approx 0.81$. The four taggings have joint probabilities 0.2688 (NV), 0.0336 (NN), 0.0168 (VN), and 0.0144 (VV), which sum to the forward result.

Learning without labels

If the hidden states were observed, as in a corpus where every word is tagged, the parameters would be estimated by counting, as in §2. When they are not, the likelihood $p(x_{1:T})$ is a sum over hidden sequences, and maximizing it has no closed form. The **expectation–maximization** (EM) algorithm alternates two steps. The E-step computes, under the current parameters, the expected number of times each transition and each emission was used, which requires the posterior probabilities of the hidden states; these come from the forward variables together with backward variables $\beta_t(i) = p(x_{t+1:T} \mid z_t = i)$, computed by the mirror-image recursion. The M-step re-estimates the parameters from these expected counts as if they were real counts. For HMMs this is the Baum–Welch, or forward–backward, algorithm (Baum et al., 1970); EM in general is due to Dempster, Laird, and Rubin (1977). Each iteration does not decrease the likelihood, but the likelihood is not concave and EM can stop at a poor local maximum. The updates are given at the end of this section, and Rabiner (1989) is the classic tutorial.

HMMs were the workhorse of speech recognition and part-of-speech tagging for two decades. They also show the limits of a small discrete state: an HMM with K states remembers at most $\log_2 K$ bits about the past, and its parameters grow as K^2.

The noisy channel

Speech recognition and machine translation were where language models first mattered commercially, through one equation. Suppose we observe a signal y (an acoustic waveform, or a sentence in French) and want the English sentence e that produced it. Treat y as the output of a noisy channel that corrupted e. By Bayes' rule,

$$\hat{e} = \arg\max_e p(e \mid y) = \arg\max_e \underbrace{p(y \mid e)}_{\text{channel model}} \underbrace{p(e)}_{\text{language model}}.$$

The decomposition splits the problem in two. The channel model says how English sounds or how it is translated, and is trained on paired data, which is scarce. The language model says which English

sentences are plausible, and is trained on English text alone, which is plentiful. Frederick Jelinek's group at IBM built speech recognizers this way from the 1970s (Jelinek, 1976), with HMMs as channel models and n-gram models as priors, and later took the same equation to translation (Brown et al., 1990; 1993). A remark Jelinek made at a workshop in 1988, in his own later wording "whenever I fire a linguist our system performance improves" (Jelinek, 2004), summarizes the shift from hand-written rules to counted statistics, and anticipates the bitter lesson of Lecture 1 (Section 4).

In the noisy channel the language model is a component: a prior that makes outputs fluent. Modern language models absorbed the other components. A translation system today is a language model conditioned on the source sentence, and the same model also does the recognition, the summarizing, and the answering.

The forward–backward algorithm

For an HMM with parameters (π, A, B) and observations $x_{1:T}$, define the backward variables $\beta_T(i) = 1$ and

$$\beta_t(i) = p(x_{t+1:T} \mid z_t = i) = \sum_j A(i,j)\, B(j, x_{t+1})\, \beta_{t+1}(j).$$

Then $p(x_{1:T}) = \sum_i \alpha_t(i)\beta_t(i)$ for every t, and the posterior quantities needed for EM are

$$\gamma_t(i) = p(z_t = i \mid x_{1:T}) = \frac{\alpha_t(i)\beta_t(i)}{p(x_{1:T})},$$
$$\xi_t(i,j) = p(z_t = i, z_{t+1} = j \mid x_{1:T}) = \frac{\alpha_t(i) A(i,j) B(j, x_{t+1}) \beta_{t+1}(j)}{p(x_{1:T})}.$$

The M-step sets the parameters to the expected relative frequencies:

$$\pi(i) \leftarrow \gamma_1(i), \qquad A(i,j) \leftarrow \frac{\sum_{t<T} \xi_t(i,j)}{\sum_{t<T} \gamma_t(i)}, \qquad B(i,v) \leftarrow \frac{\sum_{t : x_t = v} \gamma_t(i)}{\sum_t \gamma_t(i)},$$

summed over all training sequences. These are the counting estimates of §2 with expected counts in place of observed ones. That EM never decreases the likelihood follows from Jensen's inequality: for any distribution r over hidden sequences, $\log p(x) \geq \sum_z r(z) \log \frac{p(x,z)}{r(z)}$, with equality when r is the posterior. The E-step makes the bound tight at the current parameters, and the M-step maximizes the bound, so the likelihood cannot fall. In practice the α and β values underflow for long sequences and are rescaled at each step, or computed in log space.

A.7 How far back gradients reach

For the recurrent network of §7, write $\mathbf{a}_j = W\mathbf{h}_{j-1} + V\mathbf{e}_j + \mathbf{d}$, so $\mathbf{h}_j = \tanh(\mathbf{a}_j)$ and

$$J_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = D_j W, \qquad D_j = \operatorname{diag}(1 - \tanh^2(\mathbf{a}_j)).$$

Backpropagation through time computes $\mathbf{g}_j = \partial L / \partial \mathbf{h}_j$ from the last step backwards, $\mathbf{g}_{j-1} = J_j^\top \mathbf{g}_j + \partial L_{j-1} / \partial \mathbf{h}_{j-1}$, where L_{j-1} is the loss at step $j-1$; the gradient for W is the sum over steps of $(D_j \mathbf{g}_j)\mathbf{h}_{j-1}^\top$, one term per use of W. The part of $\mathbf{g}_{t-k}$ that comes from the loss at step t is $J_{t-k+1}^\top \cdots J_t^\top \, \partial L_t / \partial \mathbf{h}_t$. Since every entry of D_j lies in $(0, 1]$, $\|D_j\| \leq 1$ and

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} \right\| \leq \prod_{j=t-k+1}^{t} \|D_j\| \, \|W\| \leq \sigma_{\max}(W)^k.$$

If $\sigma_{\max}(W) < 1$, the influence of step $t-k$ on the gradient from step t decays at least geometrically in k. The converse does not hold: $\sigma_{\max}(W) > 1$ is necessary for gradients to explode, not sufficient, because saturated units have D_j near zero (Pascanu, Mikolov, and Bengio, 2013).

For the LSTM, the cell recursion $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$ gives $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \operatorname{diag}(\mathbf{f}_t)$ plus terms that pass through $\mathbf{h}_{t-1}$ and the gates. Along the direct path the product over k steps is $\operatorname{diag}(\mathbf{f}_{t-k+1} \odot \cdots \odot \mathbf{f}_t)$, which stays near the identity for units whose forget gates stay near 1. That is the sense in which the cell is a highway for gradients: its decay rate is a quantity the network chooses, unit by unit, rather than a consequence of the weights and the nonlinearity.

A.8 The gated recurrent unit

The **gated recurrent unit** (Cho et al., 2014) keeps the LSTM's central idea, an additive path whose decay the network controls, with fewer parts (Figure 24). It has no separate cell: the state $\mathbf{h}_t$ is the memory. It has two gates and a candidate:

$$\begin{aligned}
\mathbf{z}_t &= \sigma(W_z\mathbf{h}_{t-1} + V_z\mathbf{e}_t + \mathbf{b}_z) && \text{update gate: how much of each unit to replace}\\
\mathbf{r}_t &= \sigma(W_r\mathbf{h}_{t-1} + V_r\mathbf{e}_t + \mathbf{b}_r) && \text{reset gate: how much of the old state the candidate sees}\\
\tilde{\mathbf{h}}_t &= \tanh(W(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + V\mathbf{e}_t + \mathbf{b}) && \text{candidate}\\
\mathbf{h}_t &= (1-\mathbf{z}_t)\odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.
\end{aligned}$$

Compared with the LSTM of §8, three things are simpler. The update gate does the work of both the forget gate and the input gate, tied together as $\mathbf{f} = 1 - \mathbf{z}$ and $\mathbf{i} = \mathbf{z}$: each unit's new value is a weighted average of its old value and the candidate, so what is written replaces a matching share of what was kept, and the state stays between -1 and 1 without a further squashing step. There is no output gate, so the whole state is visible to the rest of the network. And the reset gate takes a new role: with $\mathbf{r}_t$ near 0 the candidate is computed from the input alone, so a unit can start afresh.

The gradient path is the LSTM's. Along the direct path, $\partial\mathbf{h}_t/\partial\mathbf{h}_{t-1}$ is $\mathrm{diag}(1-\mathbf{z}_t)$, plus terms that pass through the gates and the candidate, so a unit whose update gate stays near 0 carries its value, and its gradient, across many steps, as an LSTM cell with its forget gate near 1 does. A GRU layer has three sets of weights where the LSTM has four, $3h(h+m+1)$ parameters. Empirical comparisons found the two close: Chung et al. (2014) on music and speech, and Jozefowicz, Zaremba, and Sutskever (2015), who searched over thousands of variants, found that starting the LSTM's forget-gate biases at 1 closes the gap between the LSTM and the GRU.

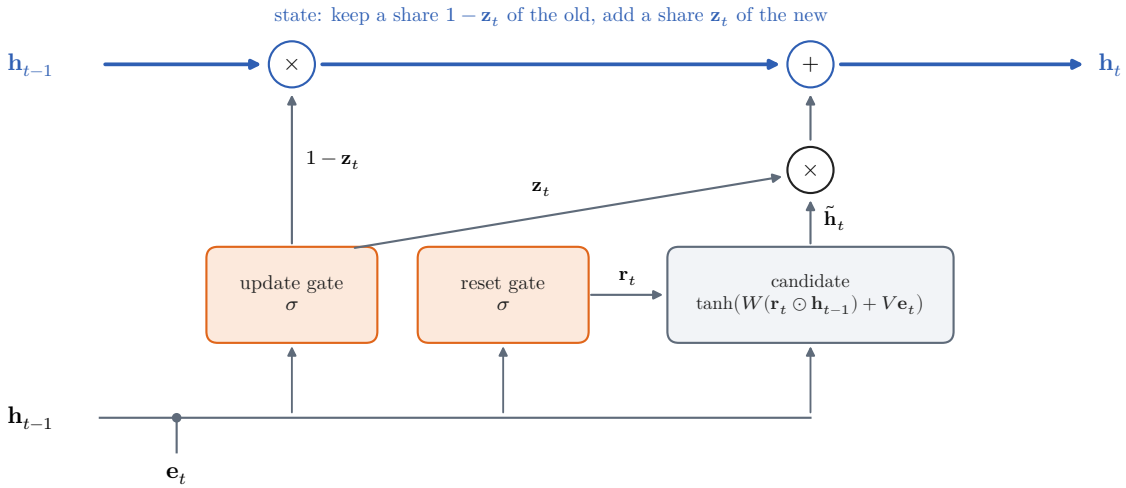


Figure 24. One step of a gated recurrent unit. The state line at the top is the memory: the update gate $\mathbf{z}_t$ decides, unit by unit, how much of the old state to keep $(1-\mathbf{z}_t)$ and how much of the candidate $\tilde{\mathbf{h}}_t$ to add $(\mathbf{z}_t)$; the reset gate $\mathbf{r}_t$ decides how much of the old state the candidate is computed from.